



TRABAJO FIN DE GRADO
GRADO EN INGENIERIA INFORMATICA

Proyecto Hopblog

Herramienta Web de animación digital

Autor

Antonio Sánchez Alcaraz

Director

Pablo García Sánchez



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2025

Proyecto HopBlog Portal de animación web

Antonio Sánchez Alcaraz

Palabras clave: *software libre, animación, flipnote studio, almacenamiento web, despliegue en la nube, base de datos, dibujo libre, procesamiento de video, fotogramas, codificación de video.*

Resumen:

En el panorama actual del dibujo y arte digital, se necesitan herramientas gratuitas o de fácil acceso que permitan al usuario medio poder practicar y aprender las bases de la animación sin tener que instalar un software de dibujo exigente para su ordenador personal o darle la oportunidad de adentrarse en el mundo sin tener que invertir una suma de dinero en suscripciones o planes de pago que a lo mejor pueden dar lugar a rechazo o descontento con el programa.

El objetivo de este proyecto es desarrollar una aplicación web para crear animaciones, almacenarlas, clasificarlas y buscarlas entre varios usuarios de la plataforma. Simulando el programa, ya discontinuado, para consolas portátiles *Nintendo DS y 3DS* llamado *Flipnote Studio*; se creará una herramienta para animar de forma intuitiva que permita dibujar en un lienzo de forma libre, es decir siguiendo el trazo definido por el ratón y creando los diferentes fotogramas que compondrá el video de salida. Asimismo permitirá almacenar las animaciones de distintos usuarios para su visualización en una galería conjunta creando una comunidad de animadores.

Same, but in English

Antonio Sánchez Alcaraz

Keywords: *open source, animation, flipnote studio, web storage, cloud deployment, database, hand free drawing, video processing, frames, video coding.*

Abstract:

The current outlook for drawing and digital art is in need of free and easy to access tools that allow the common user to practice and learn about the basics of animation without downloading demanding drawing software for their personal computer or to give them the opportunity to delve into the world without having to invest a sum of money in subscriptions or paid plans that may lead to rejection or dissatisfaction with the program.

The main goal of this project is to develop a web application for making animations, store them, sort them and find them between multiple users of the platform. Mimicking the program, now discontinued, for portable consoles *Nintendo DS y 3DS* titled *Flipnote Studio*; There will be created a tool for animating in an intuitive way allowing it to draw hand free style, in other words following the mouse path and creating multiples frames that will make up the output video. Additionally the program will allow to store animations from different users for viewing in a shared gallery making an animators community.

D. **Tutora/e(s)**, Profesor(a) del ...

Informo:

Que el presente trabajo, titulado **Chief**, ha sido realizado bajo mi supervisión por **Estudiante**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a Septiembre de 2025.

El/la director(a)/es:

(nombre completo tutor/a/es)

Agradecimientos

Agradezco la ayuda de mi tutor Pablo García por la gran cantidad de apoyo con este trabajo surgido de una pasión personal como lo es algo tan nimio como lo es un programa descargable para consolas, además de la cantidad de reuniones y tutorías que le he pedido sin poner ninguna pega y ayudándome a abarcar cualquier problema de desarrollo que ha surgido.

Agradezco también el apoyo de mis padres por cuidarme tanto cuando enfermaba mientras hacía este proyecto como si necesitaba hacer sesiones largas de implementación, gracias por confiar en mí.

No me puedo olvidar de todo el apoyo de mis amigos que me han ayudado con este proyecto, a mi amigo Pablo por haber sido la persona que más me ha ayudado tanto dándome la idea de este trabajo, como en la carrera, como en la forma de cómo tratar los estudios, es la definición de un colega. A riesgo de parecer que estoy añadiendo amigos por incrementar el número de hojas de la memoria, tampoco puedo agradecer lo suficiente el apoyo de mis amigos siempre atentos: Jesús, Sara, Inés, Germán, Néstor, Jordi, Pepe, Juan, Moisés, Claudia, *Iñaki*, Pablo y resto de amigos de más lejos como Paula, Raúl, Oscar... gracias por aguantar mis constantes agobios de no poder quedar porque tengo que estudiar, esperemos que esta sea la última vez que lo pueda decir.

Índice general

1. Introducción	17
1.1. Motivación	17
1.2. Propuesta	19
1.2.1. Funcionalidades con respecto a Flipnote	19
1.2.2. Mantenimiento de un portal web	20
1.2.3. Implicaciones de software libre	21
1.3. Estructura de la memoria	21
2. Descripción del problema	23
2.1. Propuesta del problema	23
2.2. Requisitos del proyecto	24
2.3. Objetivos a cumplir	24
3. Estado del arte	27
3.1. Software de animación digital	27
3.1.1. Historia de la digitalización	27
3.1.2. Desarrollo del arte digital	28
3.1.3. Formato de almacenamiento	29
3.2. Software similar	31
3.3. Propuestas del proyecto HopBlog	32
3.4. Aplicaciones del proyecto	32
4. Planificación y presupuesto	35
4.1. Metodología utilizada	35
4.2. Product backlog	36
4.2.1. Historias de usuario	36
4.2.2. Lista de tareas	37
4.3. Temporización	38
4.3.1. Sprint 0: Elección del lenguaje	38
4.3.2. Sprint 1	39
4.3.3. Sprint 2	40

4.3.4. Sprint 3	40
4.3.5. Sprint 4	40
4.3.6. Diagrama de Gantt	42
4.4. Presupuesto del proyecto	43
5. Análisis del problema	45
5.1. Trazo libre en un navegador	45
5.1.1. Métodos de dibujo	46
5.1.2. Fabric.js	47
5.1.3. Canvas API	47
5.1.4. Decisión final:	47
5.2. Sistemas de almacenamiento del lienzo e imágenes	48
5.2.1. Formatos de conversión	48
5.2.2. Formato escogido	49
5.3. Bases de la animación y creación de video	50
5.4. Diagrama de casos de uso	55
6. Implementación	57
6.1. Implementacion del sprint 1	58
6.1.1. Despliegue de NodeJS	58
6.1.2. Creación del lienzo	58
6.1.3. Guardado del contenido del canvas	59
6.1.4. Herramientas del lienzo	59
6.2. Implementacion del sprint 2	61
6.2.1. Paso de frontend a backend	62
6.3. Implementacion del sprint 3	64
6.3.1. Renderización de imágenes	64
6.3.2. Renderización de video	64
6.4. Implementacion del sprint 4	66
6.4.1. Sistema de usuarios	66
6.4.2. Sistema de privacidad y moderación	67
6.4.3. Medidas de seguridad y prevención a la inyección de código	68
6.5. Diagrama de clases	68
6.6. Diagrama de arquitectura	69
6.7. Diagrama de interfaces	70
6.8. Despliegue en red	71
6.8.1. Despliegue de Proyecto HopBlog	71
7. Pruebas	73
7.1. Testing del backend	73
7.1.1. Testing de páginas	74
7.1.2. Testing de funciones	77

8. Conclusiones y trabajos futuros	81
8.1. Conclusiones	81
8.2. Tareas cumplidas	82
8.3. Conclusiones del estado del arte	84
8.4. Trabajos futuros	84
9. Apéndice	87
9.1. Cómo acceder a <i>Proyecto Hopblog</i>	87
9.1.1. Acceso por navegador web	87
9.1.2. Descarga de repositorio	87
9.2. Método de uso	88

Índice de figuras

1.1.	Logo de <i>Flipnote Studio</i> para consolas <i>Nintendo 3DS</i>	18
1.2.	Interfaz de <i>Flipnote Studio</i>	20
3.1.	Comparación tasa de esquemas de compresión con la distorsión con respecto a la imagen original de una misma imagen con varios grados de compresión [16]	30
4.1.	Diagrama de Gantt de la producción del proyecto	42
4.2.	Simulación de sueldo y retención de la renta [25]	44
4.3.	Servicio de mantenimiento de base de datos de <i>Clever Cloud</i> [7]	44
5.1.	Imagen de lienzo con trazo vectorial	46
5.2.	Imagen de lienzo con trazo de mapa de bits	46
5.3.	Imagen de lienzo con trazo de mapa de bits [4]	50
5.4.	Proceso de codificación de <i>FFmpeg</i> [29]	51
5.5.	Diagrama de casos de uso de Proyecto HopBlog	55
6.1.	Lienzo del proyecto	60
6.2.	Diagrama de flujo del proceso de guardado de un video desde que se dibuja hasta que se envía al servidor	65
6.3.	Diagrama de clases del proyecto	69
6.4.	Diagrama de arquitectura	69
6.5.	Diagrama de la interfaz de las páginas	70
6.6.	Plan de servicio de <i>render.com</i> [27]	71
7.1.	Testing de página de prueba	74
7.2.	Testing de página de portada	74
7.3.	Testing de página de inicio de sesión	75
7.4.	Testing de página del lienzo	75
7.5.	Testing de usuario	75
7.6.	Testing de preview	76

7.7. Testing de subida de videos	77
7.8. Testing de creación de usuario	78
7.9. Testing de página de prueba	79
7.10. Testing de página de galería	79
7.11. Testing de función de búsqueda en la galería	79
7.12. Testing de galería total	80
7.13. Testing de borrado de galería	80

Índice de tablas

4.1. Tabla de tareas, relación entre ellas y prioridades.	38
4.2. Sprint 1	40
4.3. Sprint 2	40
4.4. Sprint 3	40
4.5. Sprint 4	41
5.1. Extensiones de H.264 [28]	53

Capítulo 1

Introducción

Esta sección nos iniciará en los conceptos a explorar en la investigación del proyecto, haciendo énfasis en la inspiración principal a la hora de desarrollar y el problema que intentamos resolver. Empezaremos con la situación actual del arte en la actualidad.

1.1. Motivación

El dibujo o sus técnicas derivadas son las corrientes artísticas más universales y accesibles que podemos imaginar, una herramienta de comunicación no verbal para manifestar una idea con un simple dibujo o esquema, ya lo dice la frase hecha “una imagen vale más que mil palabras”, la representación gráfica de una idea es el origen de múltiples doctrinas, oficios y vidas, la sociedad necesita que la gente crea y entienda imágenes.

Hoy en día se aborda la creación de imágenes de dos formas: redes neuronales, sistemas y asistentes de inteligencia artificial con capacidad de generar desde textos de forma autónoma a imágenes con un banco de referencias y alguna orden especificada, cientos de servidores dedicados a componer la definición literal de algo pedido por el consumidor intentado cumplir de la forma más lógica la petición pedida por el usuario de estas; y métodos más tradicionales, creadas, encontradas o compuestas por un humano, gente que estudia y ejerce la profesión tanto del dibujo, como edición de imágenes o fotografía, en general buscan a humanos detrás del trabajo por motivos éticos o subjetivos. Esta aplicación va orientada al público que busca la representación por humanos, dando la oportunidad a cualquiera tanto con fines lectivos como lúdicos.

La disposición de software para alguna actividad concreta determina mucho el desarrollo de esta, si proveemos de herramientas con fácil acceso, gratuitas o fácil de entender se usarán más que cualquier material profesional con múltiples capas o niveles de entendimiento casi especializado. No digo que no se requieran de herramientas especializadas, sino que recursos al alcance de cualquiera puede

mejorar la experiencia de creación, derivando en un producto más genuino o con mayor entendimiento sobre este que un asistente con inteligencia propia; los usuarios que no emplea asistentes buscan el proceso completo de crear algo, buscan el esfuerzo y los altibajos del desarrollo [30].

Muchas compañías intentan fomentar estas prácticas ofreciendo clases guiadas de sus sistemas, cursos, tutoriales dentro del software o versiones de prueba con capacidades más reducidas para entendimiento del fundamento básico del programa; si se prestan soluciones fáciles o accesibles a un usuario fomenta la práctica de esta actividad, la accesibilidad es clave a la hora de diseñar software web.

Esta aplicación web está orientada a la continuidad de la comunidad formada por los antiguos usuarios de *Flipnote Studio*, un freeware para consolas *Nintendo DS*, *DSi* y *3DS* que daba al usuario herramientas básicas de dibujo, animación y grabación de video y audio al alcance de cualquiera para el aprendizaje del medio y relación entre estos creando un círculo de artistas donde pueden cómodamente compartir su trabajo. Fue muy popular entre los usuarios debido a la sencillez de uso y accesibilidad por cualquiera.

Flipnote Studio es una herramienta de intercambio de contenido multimedia lo suficientemente completa para emplearse en otros ámbitos de desarrollo; por ejemplo se planteó como software para enseñar materias como caligrafía y pronunciación gracias a las herramientas de escritura y grabación de sonido respectivamente de la aplicación [18], dando lugar a una herramienta fácil de entender para los alumnos y de fácil acceso. También se ha empleado en el ámbito profesional para storyboard o mockup de diseño o secuencias para series de animación [13].



Figura 1.1: Logo de *Flipnote Studio* para consolas *Nintendo 3DS*

El objetivo principal de este trabajo de fin de grado es crear una plataforma o portal web para crear, subir y compartir pequeñas animaciones similares a las del software mencionado, para aquellos antiguos usuarios que ya no tienen acceso a ese programa de forma cómoda o aquellos otros que quieran iniciarse en la comunidad o aprender con esta herramienta; además, siendo un software de carácter libre puede ser expandido en utilidad por los usuarios, mejorando la calidad del servicio para otros usuarios.

1.2. Propuesta

Dado que la principal fuente de inspiración es el software de *Flipnote Studio*, haremos una clasificación de las funciones a implementar y a descartar del producto original, dado el cambio de medio.

1.2.1. Funcionalidades con respecto a Flipnote

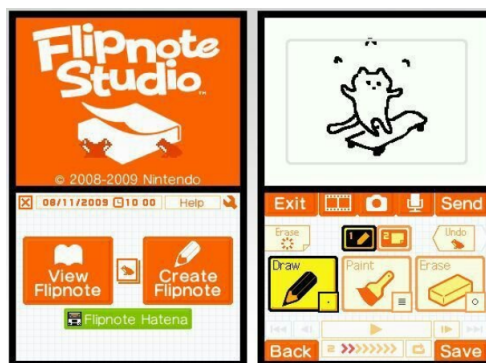
A la hora de empezar a construir software, queremos comprender las ideas que queremos que estén presentes durante todo el desarrollo, en este caso dibujar y compartir. Son ideas básicas que se pueden expandir sobre ellas una vez implementadas, estableciendo una arquitectura vertical pudiendo trabajar continuamente sobre un único proyecto para ir perfeccionándolo.

La accesibilidad puede basarse en cómo de sencillo puede ser acceder los servicios o cómo de exigente es nuestro proyecto con respecto a los requisitos del cliente, buscando el punto medio entre independencia de arquitectura y fácil instalación, abriendo el público posible que quiera emplearlo; con ello lleva el tema del rendimiento, condicionando la vida útil del programa según la carga de estrés que sufra o la extensión de uso que le damos, uno debe planear cuánto se va a usar el producto creado para poder ofrecerse al público.

Un portal web podría servir para desarrollar el software, permitiendo acceso desde cualquier punto de red y almacenando en una base de datos de un servidor o en la nube las distintas creaciones por parte de los usuarios registrados similar a un tablón de imágenes, fomentando el compartir imágenes dentro de la comunidad; citando a Tim Berners-Lee “Tendríamos que ser capaces no solo de interaccionar con otras personas, sino crear con otras personas” [11].

Teniendo ya una plataforma de desarrollo y objetivo a alcanzar, hay que buscar las herramientas que nos permitan realizar las actividades comunes de *Flipnote Studio*, no obstante es necesario aclarar que no todas las funciones del software original van a ser implementadas debido a que no aportan tanto a la idea central de animar y compartir o son aplicaciones de un uso concreto por ser la novedad en el momento que se lanzó el programa; me refiero a utilidades como el micrófono o en las versiones más modernas uso de la cámara incorporada o visión 3D de la pantalla.

Para obtener la experiencia esencial de esta aplicación, nos centramos en la implementación del lienzo de dibujo libre con tres tamaños de pincel, color a escoger para este y sobretodo capacidad de guardar la secuencia de fotogramas por separado o juntos para la conversión a video y posterior subida a la base de datos principal de la plataforma. Podríamos añadir más funcionalidades para acercarse más a la aplicación de consolas para mejorar la experiencia de dibujo o composición como pinceles de patrones, distintos niveles de capas de dibujo, deshacer la acción hecha, cambio de fondo o mover el dibujo a lo largo del lienzo; u orientado a la animación tenemos papel de cebolla para ver el contenido del fotograma anterior, velocidad de reproducción.

Figura 1.2: Interfaz de *Flipnote Studio*

1.2.2. Mantenimiento de un portal web

Crear un servicio de acceso público a contenido multimedia conlleva un acuerdo ético entre el creador y administradores pertinentes y los usuarios de esta, para cumplir unas normas estipuladas para crear una comunidad. Puede parecer sencillo dado que nuestro proyecto automatiza la creación del contenido que almacena y no da oportunidad a subir archivos que no corresponden al formato en el que se trabaja, pero obviando la instancia de inyección de código o agentes extraños, hay múltiples normas establecidas en diversas páginas web para el bien del consumo de esta, estableciendo un ambiente sano:

- Uso de etiquetas: mediante un sistema de etiquetas podemos clasificar los temas, recursos o técnicas empleada en el dibujo, para luego facilitar a todos los usuarios para la identificación de animaciones concretas para su uso y a los administradores del contenido de estas, aportando múltiples ejemplos para cada etiqueta que el usuario busque construyendo una experiencia más cómoda para todos.
- Contenido del dibujo: si el dibujo contiene algún tema adulto o no apropiado para la visualización de menores, siempre respetando las normas de la comunidad y habiendo sido aprobado por algún administrador, dando la posibilidad a contenido de cualquier edad para fomentar el uso de la aplicación.
- Material ofensivo: este proceso es muy difícil de automatizar, por ello se crean los rangos de usuarios según su nivel de control sobre la base de datos que emplea la página, siendo los administradores los que validan las entradas a la página, moderando así el contenido que es almacenado y mostrado al público general.
- Spam: el proceso de publicitar un negocio o servicio subiendo continuamente al contenido de una página ajena con publicidad de sus servicios,

no se considera ético en las páginas de intercambio de contenido al ser una técnica agresiva de auto-promoción.

- Autoría del contenido: No se puede negar la autoría de una entrada a la base de datos hecha, haciendo que el incumplimiento de las normas establecidas sea dirigida al autor de la entrada o reclamar la identidad de un archivo ya subida anteriormente para atribuirse su autoría.

1.2.3. Implicaciones de software libre

Acompañado a ofrecer un servicio a cualquiera con interés en la animación, podemos incluir la característica de software libre para tener un crecimiento vertical de nuestro software al acceso del público. A diferencia del programa *Flipnote Studio* original, que era Freeware, establecemos un proyecto de licencia libre para el dominio público de la plataforma para la contribución de cualquiera a este proyecto.

Acorde con las libertades de software libre de Richard Stallman [22] el código que engloba este proyecto software se puede usar, estudiar, distribuir y mejorar para los creadores y consumidores de este, ayudando a la comunidad.

El proyecto se recogerá en un repositorio con licencia de software libre y copyleft para clonación del repositorio de un usuario; no lo definimos como código abierto debido a la definición de las libertades centradas en el usuario que nos proporciona software libre, no tenemos intencionalidad de repartir el proyecto a organizaciones o emplear su código en otros servicios o proyectos paralelos a este, queremos hacer crecer este para aumentar las posibilidades del portal web y extender la intencionalidad y renombre del software de *Flipnote Studio*.

1.3. Estructura de la memoria

A continuación comenzaremos con el desarrollo escrito del proceso de producción de esta aplicación, dedicando cada capítulo a distintas fases de la elaboración de una plataforma web.

- Capítulo 2: una descripción de objetivo principal del trabajo y sus especificaciones, así como un desglose en tareas de menor grado para el análisis del problema.
- Capítulo 3: análisis del estado actual de la animación digital, así como los estándares y productos del mercado actuales de la industria.
- Capítulo 4: aquí explicaremos el método de trabajo así como la programación temporal y de costes del proyecto.

- Capítulo 5: investigación sobre la lógica y conocimientos aplicados para el desarrollo; escogiendo las herramientas que vamos a emplear para la creación de la plataforma web.
- Capítulo 6: análisis del código detrás del frontend y backend del software, así como una explicación de las funciones según su uso.
- Capítulo 7: apartado dedicado al *testing* y evaluación del código desarrollado.
- Capítulo 8: conclusión y declaraciones finales acerca del software creado a lo largo de los meses.
- Capítulo 9: manual o esquema sencillo de uso de la aplicación.

Capítulo 2

Descripción del problema

Como se ha comentado en el capítulo anterior, el panorama actual del arte digital ofrece múltiples alternativas para crear animaciones sin embargo tienden a ser herramientas que consumen muchos recursos, difíciles de aprender o de un coste económico elevado para alguien que quiere introducirse al mundo del dibujo digital.

El objetivo de este proyecto es crear un sustituto para el software de dibujo, grabación y animación de la saga de juegos Flipnote Studio de consolas portátiles de Nintendo DS y 3DS debido al cierre de los servidores red y páginas de descarga del programa ha sido discontinuado y carece de soporte o forma legal de adquirirlo según se escribe este documento.

- El panorama actual de los software de animación es de herramientas de pago o de alta curva de aprendizaje, dando lugar a rechazo a la gente principiante o que solo lo aprende con fines lúdicos o de forma autodidacta.
- En algunos casos, estos programas suelen ser bastante exigentes con las especificaciones del ordenador, requiriendo una buena arquitectura incluso si quieres hacer algo sencillo para iniciarse en el mundo o estudiar el medio.
- En el caso de Flipnote Studio, es un software exclusivo de consolas portátiles Nintendo DS y 3DS, limitando su distribución a poseer estos sistemas con el software preinstalado dado que cerraron los servidores de descarga online de software de las consolas mencionadas
- El problema general es la falta de accesibilidad a herramientas sencillas de animación para aprender o compartir proyectos creados.

2.1. Propuesta del problema

Dado que la principal fuente de inspiración de este trabajo es el software de Flipnote Studio, se quiere recrear una versión reducida de este para navegadores

que pueda ser accesible por cualquiera independientemente de la arquitectura del dispositivo y compartiendo entre la comunidad sus animaciones, guiones gráficos o dibujos creados por múltiples usuarios de la plataforma.

Se desarrollará un portal web para buscar entre varias animaciones para aprender o simplemente por entretenimiento, creando una plataforma para usuarios que buscan entre una base de datos de material subido por una comunidad de animadores.

Además se estudiará el mantenimiento de una base de datos y la administración con respecto al cliente para facilitar la búsqueda o la filtración del contenido de la web según las etiquetas asignadas al vídeo almacenado.

2.2. Requisitos del proyecto

El proyecto está orientado a ser desarrollado con el objetivo de ser software libre, dando la posibilidad de ser ampliado tanto por personas afiliadas al proyecto como usuarios externos que buscan mejorar la experiencia de la aplicación web sin perder la autoría de este.

El proyecto empleará recursos en su mayoría de javascript sobretodo módulos y librerías compatibles con el entorno de ejecución de Node JS para la parte del backend; servidor, acceso a base de datos y redirección de páginas y gestión de sesiones del cliente.

2.3. Objetivos a cumplir

Dado que estamos desarrollando un portal web de animaciones, necesitamos aprender el proceso de creación del mantenimiento de una página con su propio servidor web para almacenamiento y una herramienta de dibujo de trazo libre con las herramientas necesarias fácil de trabajar y almacenar (Objetivos de Aplicación, OA).

- **OA1:** Herramienta completamente online, no necesitando descargar software o cualquier suplemento de soporte este.
- **OA2:** Permita dibujar en un lienzo de forma libre.
- **OA3:** Permita dibujar varios lienzos para guardarlos en forma de video.
- **OA4:** Función donde se puedan ver todos los videos que se hayan subido a la plataforma.

Ahora relativo al desarrollo del proyecto, queremos realizar un trabajo de investigación sobre el despliegue de un portal web con múltiples usuarios con roles y filtros para la información contenida en la base de datos de forma que sea accesible para el público y clientes de esta web. Queremos cumplir algunas expectativas respecto al desarrollo web (Objetivos de Desarrollo, OD):

- **OD1:** Conexión a una base de datos estática.
- **OD2:** Múltiples usuarios con distintos roles.
- **OD3:** Sistema de renderizado de video remoto y guardado en la base de datos.
- **OD4:** Interfaz intuitiva y mínima.
- **OD5:** Entorno de desarrollo sin bugs o al menos sin errores que puedan dar lugar a la pérdida del trabajo realizado.

Capítulo 3

Estado del arte

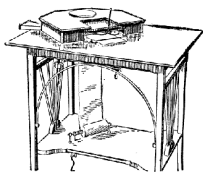
Para abordar el problema planteado necesitamos saber cómo se encuentra actualmente el panorama con respecto al dibujo y animación digital, herramientas o software que atiendan estas necesidades descritas para entender qué necesitamos o cómo plantear una solución adecuada.

3.1. Software de animación digital

El programa a desarrollar está inspirado en gran medida por el software de Flipnote Studio lanzado en 2008, sin embargo sigue las pautas del estudio de la animación con las herramientas características de un programa de animación digital, el cual su uso era bastante extendido por la época.

Esto no es un análisis del estudio de la animación como tal pero sí en el desarrollo de la animación digital como medio debido a sus influencias en las distintas funcionalidades del programa; empezando por la historia a lo largo de los años de la digitalización de trazo libre a información.

3.1.1. Historia de la digitalización



El primer antecesor de los periféricos gráficos de dibujo sería el teleautógrafo, inventado por Elisha Gray en 1888 [1] pudiendo enviar a través de corriente eléctrica escritos, copias de documentos o dibujos de líneas de trazos, realizados a mano en el momento de la transmisión, empezando así la búsqueda por poder traducir dibujo a pulsos eléctricos y viceversa; aunque usado a largas distancias como sustituto del telégrafo, dió lugar a la búsqueda del almacenamiento de imágenes gráficas en información almacenable.

A medida que avanzaba la investigación surgían diferentes connotaciones del arte digital como composiciones de formas geométricas de colores, imágenes

creadas por algoritmos o funciones matemáticas, modelados volumétricos en un entorno 3D y en los tiempos más recientes imágenes totalmente generadas de forma autónoma por inteligencia artificial; nos centramos en imágenes compuestas de píxeles de trazo libre ya sean hechas por ratón o tableta digitalizadora.

3.1.2. Desarrollo del arte digital

El primer software de dibujo o gráficos digitales, desarrollado en 1963 [14], es calificado como el primer software de manipulación directa de objetos gráficos permitiendo dibujar directamente con un lápiz óptico y almacenarlo en los archivos internos del ordenador. Este software derivará en los programas modernos de dibujo y animación.

Con el paso del tiempo se han ido desarrollando software tanto libre como de pago o uso restringido a empresas para dibujar en formato digital, listando los más famosos como son

- Microsoft Paint (1985): no fue el primer software gráfico que venía pre-instalado en un ordenador de sobremesa, precediéndole MacPaint un año antes para dispositivos Macintosh, con la diferencia de que empleaba mapa de bits específicos denominados mapas de bits o BMP que derivará en los futuros formatos de guardado de imágenes actuales [6].
- Blender (1996): herramienta conocida mayormente para diseño y creación de gráficos tridimensionales volumétricos, permitiendo esculpir, crear escenas, animarlas tanto bidimensionales como tridimensionales con la particularidad de ser open source [19].
- Clip Studio Paint (2001): software japonés de ilustración, frecuentemente asociada a la producción de historietas o cómics, toma herramientas de adobe Photoshop como inspiración para ofrecer nuevas técnicas de renderizado que no posee la competencia. Su extendido uso ha hecho que derive en versiones alternativas del software especializadas en animación u otras arquitecturas como tableta Ipad [20].
- Procreate (2011): exclusivo de iOS y demás dispositivos Apple ha aumentado en popularidad para ilustración y diseño gráfico, sirviendo como una alternativa más artística a Photoshop, siendo la primera en permitir renderizar archivos en resolución 4K y pudiendo ser utilizado incluso en móviles.

Hoy en día las empresas unifican sus softwares de animación e ilustración para abarcar un público mayor, empleándose en el ámbito profesional aun siendo free-ware, ofreciendo herramientas avanzadas en ambos campos retroalimentándose, consiguiendo así un producto mucho más completo aunque elevando la curva de dificultad de aprendizaje de este.

En 2008, se lanzó al mercado aunque fuera como software preinstalado o de descarga gratuita, Flipnote Studio para los dispositivos Nintendo DS; aprovechando la funcionalidad táctil de la pantalla de presión, permitía iniciar a los usuarios al dibujo y a la animación con herramientas limitadas pero flexibles para múltiples técnicas. Posteriormente iban revisando el software añadiendo un portal web vinculado para subir animaciones y una revisión de la aplicación en consolas posteriores.

3.1.3. Formato de almacenamiento

La creación de técnicas de digitalización de imágenes o conjuntos de píxeles requiere de un sistema de almacenamiento fiable, sin pérdidas y fácil de manipular y transportar dentro del software o incluso entre distintas arquitecturas. A medida que se desarrollan nuevas herramientas se crean nuevos tipos de fichero de datos para imágenes, aportando características distintas según lo requiera el contexto. Veamos algunas:

- **Imágenes rasterizadas:** Son aquellas estructuras de datos que representa una rejilla rectangular de píxeles o puntos de color denominada matriz, suelen estar definidas por alto y ancho para determinar la resolución de la imagen que forman [17]. Hay distintos tipos de mapas dependiendo de su modelo de color, según qué colores primarios tomen RGB (Rojo, Verde, Azul), CMYK (Cíán, Magenta, Amarillo y Negro) o LAB o el algoritmo de compresión que empleen con el objeto rasterizado, veamos los tipos más comunes:
 - **BMP mapa de bits (1987):** formato de imagen nativo de Windows pero ya extendido a todas los sistemas operativos, archivos compuestos por direcciones asociadas a códigos de color en una matriz de píxeles.
 - **JPEG (1992):** es un estándar de compresión creado por la Joint Photographic Experts Group, de ahí su nombre abreviado; emplea un algoritmo con pérdidas (transformada del coseno discreta) de compresión de imagen basado en el fenómeno visual del ojo humano de ser más sensible al cambio en la luminancia que en la crominancia, centrándose así en el brillo en vez de una gran cantidad de colores por cada imagen. Sin embargo no emplea canal alfa para transparencias y su balance pérdida de calidad-tamaño puede ser mejorable [15].
 - **JPEG 2000:** creado por el mismo comité que JPEG, es una versión revisada que emplea tanto algoritmo de la transformada del coseno discreta como otros basado en wavelets. Puede conservar mucho mejor la imagen original a pesar del grado de compresión que se aplique con respecto a jpeg original 3.1.3 , sin embargo está en desuso por la popularidad de jpeg [15].

- PNG Portable Network Graphics (1996): a diferencia de los anteriores, emplea un algoritmo de compresión sin pérdida basado en bitmaps. Contienen una cabecera de firma de 8 bytes que indica las características de este archivo, seguido de diferentes secciones de los colores e iluminación de estos y finalmente se le añaden metadatos de para información textual o valores de gamma; debido al algoritmo puede ocupar más que otros formatos sin embargo no hay problema para imágenes sencillas o generadas de un mapa de bits canvas como planeamos en este proyecto.

Imagen	Tasa	Distorsión	Imagen	Tasa	Distorsión
bird5.jpeg	1.8640 14e-01	8.613794 e+01	bird40.jp2	2.10205 1e-01	1.607787e+ 01
bird10.jpeg	2.3730 47e-01	3.999619 e+01	bird38.jp2	2.19726 6e-01	1.525677e+ 01
bird15.jpeg	2.7954 10e-01	2.646747 e+01	bird36.jp2	2.29003 9e-01	1.445131e+ 01
bird20.jpeg	3.1542 97e-01	2.049785 e+01	bird34.jp2	2.44018 6e-01	1.328276e+ 01
bird25.jpeg	3.5278 32e-01	1.668388 e+01	bird32.jp2	2.59155 3e-01	1.221671e+ 01
bird30.jpeg	3.8745 12e-01	1.441661 e+01	bird30.jp2	2.72949 2e-01	1.130623e+ 01
bird35.jpeg	4.1931 15e-01	1.269067 e+01	bird28.jp2	2.95288 1e-01	1.021249e+ 01
bird40.jpeg	4.4885 25e-01	1.160602 e+01	bird26.jp2	3.04321 3e-01	9.852234e+ 00
bird45.jpeg	4.8144 53e-01	1.051474 e+01	bird24.jp2	3.37158 2e-01	8.582962e+ 00
bird50.jpeg	5.1135 25e-01	9.637192 e+00	bird22.jp2	3.65844 7e-01	7.539917e+ 00
bird55.jpeg	5.3881 84e-01	8.899490 e+00	bird20.jp2	4.09179 7e-01	6.514191e+ 00
bird60.jpeg	5.7788 09e-01	8.169754 e+00	bird18.jp2	4.53002 9e-01	5.771332e+ 00
bird65.jpeg	6.2756 35e-01	7.352264 e+00	bird16.jp2	4.99511 7e-01	5.106918e+ 00
bird70.jpeg	6.9226 07e-01	6.519196 e+00	bird14.jp2	5.34790 0e-01	4.749557e+ 00
bird75.jpeg	7.7600 10e-01	5.724167 e+00	bird12.jp2	6.44287 1e-01	3.728180e+ 00
bird80.jpeg	9.0417 48e-01	4.823196 e+00	bird10.jp2	7.71484 4e-01	3.137268e+ 00
bird85.jpeg	1.0911 87e+00	3.866379 e+00	bird8.jp2	9.36523 4e-01	2.536942e+ 00
bird90.jpeg	1.4232 18e+00	2.811798 e+00	bird6.jp2	1.27856 4e+00	1.770126e+ 00
bird95.jpeg	2.1556 40e+00	1.596161 e+00	bird4.jp2	1.93896 5e+00	8.827820e- 01
bird100.jpeg	4.2150 88e+00	9.368896 e-02	bird2.jp2	3.14221 2e+00	2.653351e- 01

Figura 3.1: Comparación tasa de esquemas de compresión con la distorsión con respecto a la imagen original de una misma imagen con varios grados de compresión [16]

- Imágenes vectorizadas: Son imágenes digitales formadas exclusivamente por figuras geométricas dependientes los cuales tienen atributos definidos

de tamaño, longitud, radio, etc; permiten ampliar el tamaño de una imagen sin pérdida de calidad así como aplicar transformaciones de forma sencilla con solo cambiar los atributos que las conforman. [17]

- PostScript (1984): enfocado para la impresión de este archivo ya sea de texto o imagen, permitiendo figuras precisas como líneas rectas, píxeles sueltos o curvas de Bezier.
- AI (1987): desarrollado para Adobe Illustrator, puede ser usado como medio de almacenamiento de proyectos de edición o dibujo, además teniendo compatibilidad con el resto de softwares de adobe para su postprocesado. También es conocida por el uso de relleno de malla o malla de degradado que consiste en líneas vectoriales horizontales y verticales, entrecruzadas entre sí, formando una cuadrícula que permite aplicar cambios suaves de color en el relleno de un objeto cerrado, dando como resultado un degradado suave y preciso.
- SVG (2001): definidos como archivos de texto XML, mejorando su compatibilidad con diferentes software debido a su sencillez y dándole un tamaño total de archivo menor. Es empleado para iconos o imágenes para páginas web y navegadores.
- SAI (2004): especializado en el software del mismo nombre, se centra en almacenar un proyecto o dibujo sin pérdidas para su procesamiento o continuarlo después, puede conservar con gran nivel de detalle trazos, colores y múltiples capas de una imagen sin distorsión de esta ni pérdida de información.

3.2. Software similar

Gracias a la mejora del soporte en la nube, infraestructura de envío de datos, ancho de banda mejores y demás avances tecnológicos permiten la capacidad de compartir y crear dibujos o animaciones online similar al objetivo de este proyecto. Vamos a ver algunas páginas que he tomado como modelo a seguir para desarrollar el proyecto.

- Clipnote: es un freeware de animación que intenta replicar las herramientas y técnicas empleadas en Flipnote Studio como 6 colores formando distintos pinceles formados por píxeles, capas y descarga de las animaciones realizadas. Es una herramienta creada por fans del programa por un equipo reducido para continuar su servicio, sin embargo el proyecto fue abandonado a pesar de que quedan bastantes funciones por implementar que pide el público [5].
- Sudomemo: es un portal web para subir animaciones 'flipbook' creadas y publicadas desde consolas Nintendo DSi y 3DS en flipnote studio. Es una

continuación del servicio de Flipnote Hatena, sirviendo como herramienta auxiliar en la web para conservar animaciones y compartirlas [21].

- Wigglypaint: es una aplicación web de dibujo con la característica de que el producto resultante es un archivo en formato gif dado que el trazo del dibujo está siempre en movimiento vibrando, dándole más personalidad. Es una herramienta sencilla desplegada tanto en itch.io como en su propia página web, desarrollado en la plataforma de Decker, especializada en recursos multimedia [10].
- Gartic Phone: es un juego web multijugador con múltiples modos todos basándose en el dibujo en tiempo real entre varias personas para formar una obra conjunta. Tiene varios modos de juego, de especial interés el modo de animación donde cada uno dibuja un frame de una pequeña animación luego renderizada en formato GIF animado [23].

3.3. Propuestas del proyecto HopBlog

Hemos visto una comunidad dedicada a la conservación del público de Flipnote Studio sin embargo no se ha visto reproducido en su totalidad o se requiere de múltiples herramientas simultáneas para su simulación. En este proyecto buscamos:

- Se quiere una herramienta sencilla, con herramientas básicas pero populares de dibujo para inicializar al público sin sentirse abrumado por las opciones.
- Permita crear, almacenar y compartir animaciones en un mismo portal web.
- Sistema de login de usuarios, solo permitiendo crear animaciones a los usuarios identificados para siempre tener autoría del material subido.
- Ofrecer una página, sea usuario registrado o no, de visionado de animaciones subidas a la plataforma.

3.4. Aplicaciones del proyecto

El desarrollo de este portal web está enfocado a todo lo relativo a la animación y preservar o crear un sustituto de *Flipnote Studio*, sin embargo este programa originalmente sirvió para múltiples aplicaciones tangentes a este ámbito. Se presentan algunas:

- Educación: puede emplearse como herramienta de iniciación tanto al mundo del dibujo digital y animación como inicio a las tecnologías TIC de comunicación y subida de recursos a la web [18].

- Recreación: *Flipnote Studio* es originalmente un programa perteneciente a consolas portátiles de videojuego como son la Nintendo DS y Nintendo 3DS, ofreciendo métodos de entretenimiento y libertad creativa a la hora de hacer animaciones o ver las de otros usuarios.
- Desarrollo multimedia: *Flipnote Studio* ha sido empleada como herramienta rápida de trabajo para realizar guiones gráficos o visualizar la disposición de los elementos de animaciones de toda clase dado su fácil uso o sencillo enfoque de cómo funciona una animación. Los guiones gráficos están a la orden del día para cualquier tipo de entretenimiento visual ya sea animado o con actores reales [13].

Capítulo 4

Planificación y presupuesto

4.1. Metodología utilizada

No se puede empezar a construir una casa por el tejado, para crear un proyecto se necesita seguir una serie de pasos y procedimientos para no perder el objetivo principal de este, su público objetivo o las características con las que se quiere contar para satisfacer las necesidades del cliente y del desarrollador. Esto no es diferente a la hora de crear un proyecto software, se han desarrollado múltiples filosofías y metodologías de trabajo que ayudan al creador, integrantes del equipo y gente de terceros a entender la finalidad y construcción detrás de un programa.

Hoy en día las metodologías más populares puedes agruparse en dos tipos de enfoques: el enfoque **clásico**: lineal, secuencial y creciente; empieza con un análisis de la finalidad del proyecto, casos de uso, requisitos funcionales, presupuesto... todo lo que se desarrolle a posteriori será relativo a la planificación de un proyecto de antemano. Después del periodo de organización y puesta en común se empieza con el trabajo práctico (ya sea tanto diseño de algoritmos a emplear, funciones, bases de datos o incluso interfaces) y su implementación y desarrollo para luego someterse a un periodo de pruebas del software implementado.

Sin embargo, con este proyecto hemos querido emplear otro enfoque, la metodología ágil *SCRUM*, establecida en 2001 por un equipo de 17 desarrolladores tiene como objetivo un proceso en el que se aplican de manera regular un conjunto de buenas prácticas para trabajar en equipo y obtener el mejor resultado posible [3]. Se ha escogido esta metodología debido a que está orientada a equipos pequeños o **individuales** donde se prioriza las necesidades del cliente para el cumplir los requisitos establecidos con plazos de tiempo en bloques cortos y fijos. Para ello seguimos los procesos de:

- **Progreso secuencial**: Adoptar una estrategia de desarrollo incremental, en lugar de la planificación y ejecución completa del producto.

- **Valoración de la petición del cliente:** Basar la calidad del resultado más en el conocimiento tácito de las personas en equipos autoorganizados, que en la calidad de los procesos empleados.
- **Cohesión entre las distintas partes del proyecto:** Solapar las diferentes fases del desarrollo, en lugar de realizar una tras otra en un ciclo secuencial o en cascada.

Existen otros tipos de metodologías ágiles como *Kanban* o *Extreme Programming* que podrían haberse optado para desarrollar este trabajo sin embargo se han descartado debido a su enfoque en equipos de mayor tamaño y especialización en el desarrollo de código; abandonando un poco todo lo relacionado a documentación y memoria, se considera que la metodología *SCRUM* de tareas y sprints puede ayudarnos a la organización del proyecto.

4.2. Product backlog

Antes de empezar con los sprints de desarrollo, necesitamos especificar tanto los puntos a cumplir como los deseos del cliente a satisfacer con este proyecto, acordando así los objetivos en forma de una lista de tareas desglosables en peticiones menores para mejor organización.

4.2.1. Historias de usuario

Para la planificación del desarrollo necesitamos una idea concisa de qué queremos que haga el proyecto y para quién va destinado, esto nos ayudará para empezar el *sprint 0* de nuestra metodología *SCRUM*. Representamos del público o clientes del software con historias de usuario que nos dan una descripción del producto a diseñar, aplicaciones o instancias de uso ya sea una persona individual o un colectivo de gente o corporativo que puede emplear nuestro software; veamos algunas de ejemplo:

- **HU-1:** Antonio Fernandez, estudiante universitario con tiempo libre que busca alguna forma de crear animaciones o buscar referencias de estas. Le gustaría en su tiempo libre aprender a dibujar en formato digital, para ello busca tener acceso a una herramienta que no requiera de múltiples descargas de software pesado o exigente debido a que usa el ordenador para el estudio. Se ha planteado múltiples plataformas que ofrecen servicios de pago con una gran cantidad de herramientas, como *Clip Studio Paint* o *Adobe After Effects* y funcionalidades que no le sirven mucho para empezar a aprender de forma autodidacta y añaden un precio extra al producto total, busca una forma de practicar sin gastar dinero en licencias que a lo mejor luego no quiera emplear definitivamente debido a que no le guste la herramienta o simplemente porque no vea rentable gastar tanto capital en un hobby paralelo a su vida laboral.

- **HU-2:** Monkey's Studio es una empresa que se centra en grabaciones de anuncios en múltiples medios, ya sea animación, actores reales o cualquier medio de expresión. Para la puesta en escena necesitamos algún tipo de story-board o guión gráfico para saber cómo deberíamos filmarlo o los elementos de este. A favor de los creativos, se busca una herramienta sencilla y fácil de enseñar entre compañeros para hacer las modificaciones necesarias o fácil envío del producto a grabar. Necesitan una herramienta sencilla para un guión gráfico claro del anuncio a grabar, permita múltiples fotogramas y fácil almacenamiento de este guión; además que pueda reproducirse en cualquier ordenador de sobremesa o portátil del estudio.
- **HU-3:** Jacinta Flores es una profesora de Bellas Artes que necesita instruir un mínimo de horas en los fundamentos sobre la animación fotograma a fotograma orientada al uso de interfaces digitales. Podría emplear programas con servicio de licencia como *Clip Studio Paint* o *Toon Boom Animation* sin embargo requieren sus servicios orientados al ámbito educativo son de tiempo limitado y ligados a la institución que los solicita dando lugar a un proceso grande administrativo por cada clase que tenga que instruir en ese temario. La profesora busca una alternativa gratuita, sencilla y sin demasiados costes de hardware para practicar los conceptos básicos de animación sin depender de múltiples licencias o suscripciones por cada integrante de la clase.

Teniendo ya noción de lo que busca el público que usará esta aplicación, podemos elaborar una idea preconcebida de las funcionalidades, características y tareas a cumplir a la hora de desarrollar el código de nuestro software, pudiendo repartir estas tareas en trabajos individuales que llamaremos sprints. Estos sprints son ciclos de trabajo de duración fija, dependiendo de la complejidad o prioridad de la tarea, durante el cual el equipo desarrolla un incremento de producto potencialmente funcional y valioso hasta conseguir un prototipo usable.

4.2.2. Lista de tareas

Se ha organizado una lista de tareas a realizar para contentar las necesidades de los clientes, organizadas por título, prioridad y un sistema de referencias las cuales indican si una tarea puede derivar de la investigación de otra o si esta depende encarecidamente de la implementación suya. Respecto a la prioridad denotamos las de prioridad alta como indispensables para mantener la finalidad del software o para su correcto uso, medio para indicar que es la idea principal detrás de la tarea pero puede haber alternativas a cómo abarcarla y baja para detalles de pureza pero que pueden aumentar la calidad del servicio de la herramienta.

Deducido del apartado anterior, tenemos que el objetivo de este proyecto debe cubrir a grandes rasgos:

- Permitir dibujar de forma libre fotograma a fotograma y generar un video.
- Todo el proceso de crear una animación se pueda ejecutar en un navegador.

Y aquí tenemos la lista de tareas elaborada:

Lista de tareas			
Nº	Ref	Título	Prioridad
1	RF-1	Despliegue servidor node	Alta
2	RF-2	Lienzo de dibujo	Alta
3	RF-2	Herramientas del lienzo	Baja
4	RF-2	Guardado del lienzo	Alta
5	RF-3	Paso de información de frontend a backend	Alta
6	RF-4	Creación de base de datos	Alta
7	RF-3	Almacenamiento de lienzo en base de datos	Alta
8	RF-5	Renderización de imágenes	Alta
9	RF-6	Renderización de video	Alta
10	RF-4	Almacenamiento de video	Media
11	RF-7	Galería de visualización	Alta
12	RF-8	Sistema de usuarios	Media
13	RF-8	Sistema de privacidad y moderación	Baja
14	RF-9	Medida de seguridad de inyección de código	Media

Tabla 4.1: Tabla de tareas, relación entre ellas y prioridades.

4.3. Temporización

Vamos a repartir las tareas en los sprints correspondientes, tenemos que tener en cuenta la prioridad de las tareas a la hora de definirlos en un periodo de tiempo concreto, a continuación cada uno a fondo:

4.3.1. Sprint 0: Elección del lenguaje

El *sprint 0* es un preámbulo para los siguientes donde se escoge cómo se va a trabajar o con qué materiales se puede emplear para empezar a escribir código; habiendo ya establecido las tareas a cumplir para el objetivo a largo plazo se necesita saber con qué se va a trabajar, las sesiones y su duración de estos sprints. Se ha englobado en un mismo sprint las tareas de características similares o que empleen las mismas tecnologías para economizar la búsqueda de información al respecto; empleando funciones de forma incremental de las

tareas implementadas antes para un crecimiento vertical del proyecto. Se estima unos 4 sprints de mínimo 2 semanas cada uno para la implementación de la infraestructura, sin contar la depuración o testeo de código posterior, priorizando las tareas de alta relevancia para la existencia de portal web y funcionalidad principal y tener cuanto antes un producto funcional.

Respecto al lenguaje a emplear, este proceso requiere de un periodo variable de investigación para averiguar la herramienta que mejor se ajusta a nuestros criterios establecidos pudiendo hacer que la lógica detrás de una función específica pueda ser fácilmente implementada con una librería del lenguaje o creada de cero debido a la falta de documentación o soporte a esta. Se ha escogido el uso de *Javascript*, su extendido uso además de características intrínsecamente ligadas al desarrollo web y exploradores web lo hace mucho más cómodo y sencillo de usar.

Para el tratamiento de imágenes y creación de videos se emplea el framework de *ffmpeg*, un conjunto de librerías que permite la conversión y edición de archivos de audio y video, fue desarrollado en Linux pero hoy en día es apto para cualquier sistema operativo estándar. Se usa la librería de *JavaScript* de *ffmpeg-static* y *fluent-ffmpeg* la cual cuenta con versiones actualizadas y revisadas periódicamente afianzando la fiabilidad de esta.

La parte del backend será lanzada y soportada en *Node.js* al ser un entorno de ejecución asíncrono de uso extendido cuenta con gran soporte y documentación para cualquier situación que surja, siendo además de intuitivo y de fácil uso podemos emplearlo junto al framework de *expressjs* para mayor comodidad y limpieza de código, dándonos un mantenimiento del servidor organizado y más fácil de trabajar, tanto a la hora de crear nuevos recursos como para añadir sistemas adicionales como son las sesiones de usuario o una capa de seguridad adicional a todo lo que comprende el backend.

Respecto al manejo de la base de datos, *MySQL* permite la gestión de esta a través de la librería de *mysql2* y gracias a la simplificación aportada por el framework de *Express* podemos hacer peticiones a la base de datos remota de forma cómoda y sencilla con un sistema de filtrado que evita la inyección de sentencias SQL pudiendo abrir una brecha de seguridad en la permanencia de datos de los usuarios de la página.

4.3.2. Sprint 1

Primer sprint de desarrollo de la aplicación, aquí haremos por separado el despliegue del servidor y la creación del lienzo y cómo acceder a su contenido, cubriendo las dos ideas sacadas de las historias de usuario. **Duración estimada: 2 semanas.**

Ref	Título	Prioridad
RF-1	Despliegue servidor node	Alta
RF-2	Lienzo de dibujo	Alta
RF-2	Guardado del lienzo	Alta
RF-2	Herramientas del lienzo	Baja

Tabla 4.2: Sprint 1

4.3.3. Sprint 2

Nos centramos en el paso de frontend a backend, es decir el paso de información cliente a servidor y viceversa; se necesita entender cómo tratar los datos, cómo pasarlos y saber la mejor forma de almacenarlos en el servidor. **Duración estimada: 6 semanas.**

Ref	Título	Prioridad
RF-3	Paso de información de frontend a backend	Alta
RF-4	Creación de base de datos	Alta
RF-3	Almacenamiento de lienzo en base de datos	Alta

Tabla 4.3: Sprint 2

4.3.4. Sprint 3

Teniendo almacenadas las imágenes y pudiéndolas pasar de cliente a servidor, necesitamos crear un video de estas y enviársela al cliente para completar el proceso. **Duración estimada: 3 semanas.**

Ref	Título	Prioridad
RF-5	Renderización de imágenes	Alta
RF-6	Renderización de video	Alta
RF-4	Almacenamiento de video	Media

Tabla 4.4: Sprint 3

4.3.5. Sprint 4

Dado que estamos haciendo una herramienta web, habría que implementar un sistema de autoría de las imágenes creadas y guardadas en la base de datos, así como una capa de seguridad para evitar el sabotaje de la información almacenada. **Duración estimada: 3 semanas.**

Ref	Título	Prioridad
RF-8	Sistema de usuarios	Media
RF-6	Sistema de privacidad y moderación	Baja
RF-4	Medidas de seguridad de inyección de código	Media

Tabla 4.5: Sprint 4

4.3.6. Diagrama de Gantt

Aquí vemos el reparto aproximado del desarrollo a lo largo del tiempo de producción del proyecto.

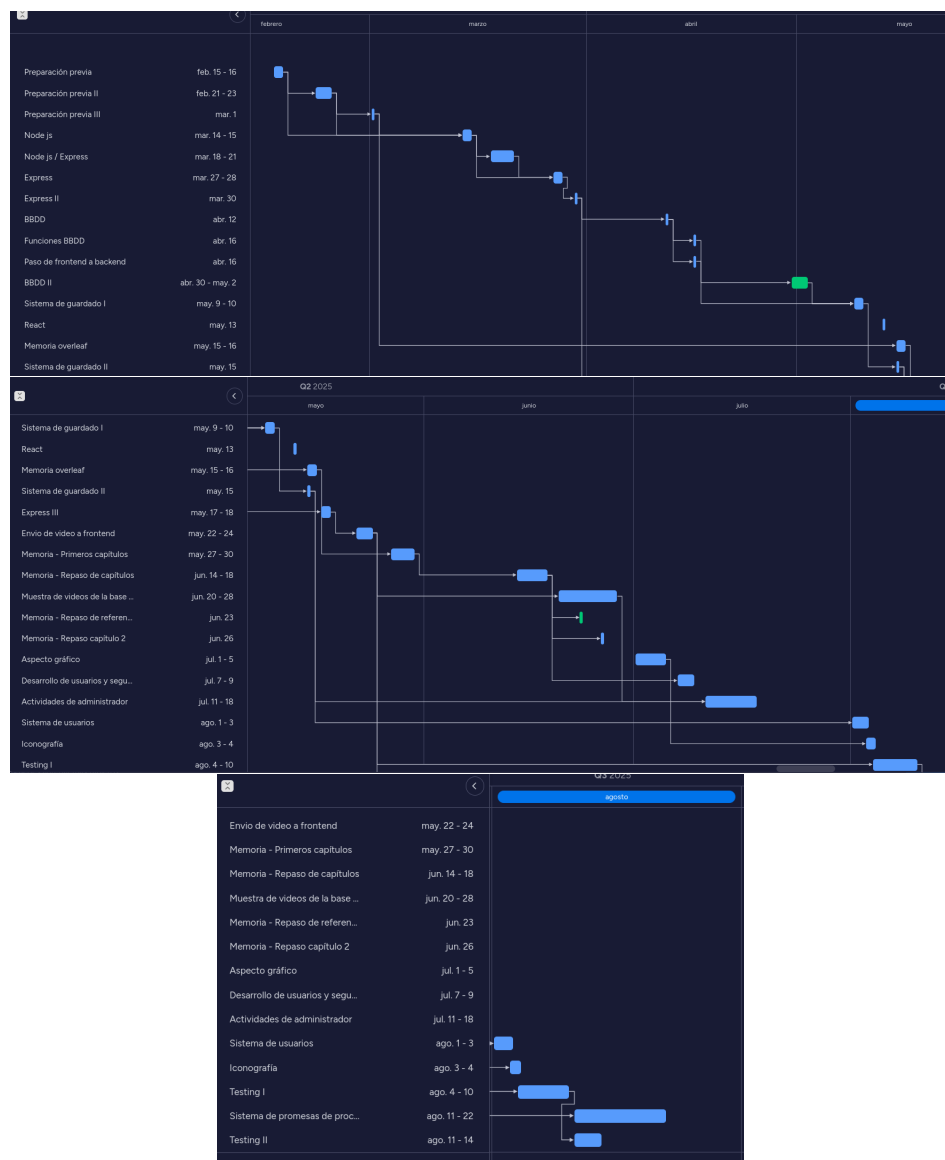


Figura 4.1: Diagrama de Gantt de la producción del proyecto

4.4. Presupuesto del proyecto

El capital invertido y múltiples valores empleados del presupuesto son estimaciones basadas en el estándar medio de la industria, el resultado real puede variar debido a que el trabajo realizado no es un proyecto real para lanzar al mercado. Veamos una estimación del valor de costes de producción.

Hacemos un acopio de los datos relevantes para el cálculo del presupuesto:

- Trabajo de aproximadamente 4 meses de desarrollo del producto, estimando sesiones de 1 a 3 horas, en algunos casos más dependiendo de la semana o la urgencia de la tarea, se establecerán unas 230 horas de desarrollo de código.
- Añadido el tiempo posterior de testeo y despliegue de la aplicación aproximadamente de 23 horas en dos semanas, sale a un total de 253 horas totales. Consultando en páginas como *Glassdoor* [9], el sueldo medio de un programador web es de 24000 euros al año, teniendo en cuenta que la media de días laborales en España es de 249 [26] con jornadas de 8 horas da un total de **12 euros por hora**. Aplicándose a nuestras horas tenemos un total de **3036€** por el desarrollo del proyecto.
- Si contamos el valor de *IRPF* de España en 2025 con una retención del 10% y 800€ destinados a la *Seguridad Social*, de forma simulada nos saldría una retención de
- El hardware empleado ha sido un portátil *VivoBook-ASUSLaptop-X515JA-F515JA* haciendo una media entre varias páginas sale un valor aproximado de **529€**.
- Respecto al software no se ha empleado software de pago en ninguna instancia del proyecto, en un hipotético caso del mantenimiento de la base de datos en la nube, según la página de *Clever Cloud* [7] tenemos precios según la capacidad de almacenamiento, desde 5 a 50 euros. Simulemos la oferta mínima del plan de **5.90€**.

Teniendo en cuenta todo lo anterior podemos hacer una tabla con el importe total de desarrollo del proyecto:

Concepto	Coste en euros
Salario (4 meses)	3036€
Retención IRPF	1007.45€
Material hardware	529€
Material software	0€
Material oficina	20€
Mantenimiento BBDD	23.6€
Coste total: 4616,05€	

Determinación de cuotas líquidas y resultados		
CUOTAS LÍQUIDAS		
Cuota líquida estatal	1.735,50	[0570]
Cuota líquida autonómica	1.671,95	[0571]
Cuota líquida estatal incrementada [(570)+(568)+(582)+(572)+(573)+(574)+(576)]	1.735,50	[0585]
Cuota líquida autonómica incrementada [(571)+(569)+(583)+(577)+(578)+(579)+(581)+(584)]	1.671,95	[0586]
CUOTA RESULTANTE DE LA AUTOLIQUIDACIÓN		
Cuota líquida incrementada total [(585)+(586)]	3.407,45	[0587]
Cuota resultante de la autoliquidación [(587)-(588)-(589)-(590)-(591)]	3.407,45	[0595]
RETENCIONES Y DEMÁS PAGOS A CUENTA		
Por rendimientos del trabajo	2.400,00	[0596]
Total pagos a cuenta [suma de (592) + (593)+ (594) + (596) a (606)]	2.400,00	[0609]
RESULTADO DE LA DECLARACIÓN		
Cuota diferencial [(595)-(609)]	1.007,45	[0610]
Resultado de la declaración	1.007,45	[0670]
Importe del IRPF que corresponde a la Comunidad Autónoma de residencia del contribuyente		
Cuota líquida autonómica incrementada	1.671,95	[0671]
Importe del IRPF que corresponde a la Comunidad Autónoma de residencia del contribuyente	1.671,95	[0675]

Figura 4.2: Simulación de sueldo y retención de la renta [25]

clever cloud							
Presentation Products Solutions Resox							
Plan	vCPUs	RAM	Disk size	Connection limit	Logs	Metrics	Price/30 days
DEV	Shared	Shared	10 MIB	5	No	No	€0.00 +
XXS Small Space	1	512 MIB	512 MIB	15	Yes	Yes	€5.00 +
XXS Medium Space	1	512 MIB	1 GIB	15	Yes	Yes	€5.90 +
XXS Big Space	1	512 MIB	2 GIB	15	Yes	Yes	€6.80 +
XS Tiny Space	1	1 GIB	2 GIB	75	Yes	Yes	€15.00 +
XS Small Space	1	1 GIB	5 GIB	75	Yes	Yes	€23.50 +
XS Medium Space	1	1 GIB	10 GIB	75	Yes	Yes	€26.00 +
XS Big Space	1	1 GIB	15 GIB	75	Yes	Yes	€30.00 +
S Small Space	2	2 GIB	10 GIB	125	Yes	Yes	€50.00 +
S Medium Space	2	2 GIB	15 GIB	125	Yes	Yes	€52.50 +

Figura 4.3: Servicio de mantenimiento de base de datos de *Clever Cloud* [7]

Capítulo 5

Análisis del problema

5.1. Trazo libre en un navegador

Para desarrollar una herramienta de animación online necesitamos una manera de capturar los movimientos del ratón pulsado a un lienzo para simular una pizarra o lienzo para dibujar un fotograma, *JavaScript* al ser un lenguaje tan intrínsecamente enlazado al navegador web tiene varias formas de tomar la posición global del ratón en la pantalla dentro de la pestaña de la aplicación dando lugar a múltiples librerías, funciones o elementos específicos de dibujo o representación gráfica de figuras que ayude a representar el trazo libre a esbozar. Hay que tener en cuenta que realmente estamos simulando un trazo continuo, dado que estará formado por figuras o elementos gráficos contiguos dando la ilusión de una línea, dando lugar a dos tipos de lienzos que afectará directamente en su forma de almacenarlos:

- **Trazo vectorial:** una línea está formada por pequeñas figuras seguidas una detrás de otra siguiendo la trayectoria del ratón al ser pulsado, es decir en realidad son figuras geométricas añadidas al lienzo (generalmente círculos o cuadrados dependiendo del acabado del trazo) que simulan una línea contigua. Este método puede ser ideal a la hora de almacenar el contenido debido a que ocupa menos que su contraparte sin embargo depende de la velocidad de trazado y capacidad de recogida de información debido a que si un usuario mueve demasiado rápido surgen espacios en blanco entre figuras pudiendo alterar el resultado que busca este.
- **Mapa de bits:** las líneas son representadas por los bits que conforman el espacio del lienzo, coloreándose según el ratón pasa pulsado sobre ellos. Este método da un trazo mucho más contiguo y natural y se acerca mucho más al del producto original que inspira el proyecto. Se ha tomado este método para proceder con el desarrollo de la aplicación.



Figura 5.1: Imagen de lienzo con trazo vectorial



Figura 5.2: Imagen de lienzo con trazo de mapa de bits

Con la idea de emplear mapa de bits para el lienzo seguimos la investigación de cómo dibujar sobre un lienzo en un entorno web.

5.1.1. Métodos de dibujo

JavaScript permite múltiples formas de acceder a los elementos de un archivo html tanto de forma real como asíncrona lo cual facilita bastante el desarrollo de librerías o funciones de interacción cliente-página; por ello se han desarrollado múltiples frameworks o entornos de dibujo para el desarrollo web, analizaremos unos cuantos para decidir cuál puede ser el mejor para implementar orientado a la animación o dibujo consecutivo de fotogramas.

5.1.1.1. KineticJS

Es un framework de *JavaScript* y Canvas HTML5 que facilita la interactividad 2D con aplicaciones móviles y de sobremesa. Se estuvo probando en las primeras instancias del proyecto para la implementación de un lienzo dibujable

debido a sus capacidades de transiciones, testeo de nodo, capas filtros y manejo de eventos de los elementos del lienzo [8]; sin embargo fue descontinuada y eventualmente dejó de mantenerse su repositorio y el proyecto entero explicado así en el repositorio del autor, también aclara que la herramienta de *Concrete.js* puede servir de alternativa pero también se encuentra descontinuada o directamente ya no presenta soporte. Para evitar posible carga de trabajo futura se descarta este framework para el desarrollo del proyecto.

5.1.1.2. Konva

Konva es una librería de *JavaScript* orientada a la creación de gráficos complejos y composiciones de lienzo usando *Angular* o *React*. Según su repositorio [12] esta herramienta empezó como un *fork* del repositorio original de *KineticJS* pero ha ido desarrollando su propias funciones más orientadas a la creación de imágenes compuestas de otros elementos gráficos a modo de *collage* de forma interactiva. Tiene implementaciones nativas en frameworks de *JavaScript* como se ha comentado sin embargo se tiene que descartar debido a que está más orientado al uso de imágenes superpuestas más que al trazo libre o recolección de vector de puntos por lo que se descarta esta opción.

5.1.2. Fabric.js

Una librería para Canvas HTML5 y *JavaScript* que provee de un modelo de interacción de objetos encima de elementos de lienzo HTML además de un conversor SVG(dibujo vectorial) a *Canvas* y viceversa. En la documentación oficial de la librería aclara que no todos los elementos posibles de introducir en un canvas puedan mantenerse igual si se pasa de canvas a imagen vectorial, dando lugar a posibles pérdidas del fotograma originalmente dibujado, evitamos ese riesgo descartando la herramienta.

5.1.3. Canvas API

Gracias a la posibilidad de interactuar directamente con objetos de *JavaScript* podemos interactuar directamente con el contenido de un elemento concreto de la página html, en este caso el elemento canvas y guardar en un vector de elementos el contenido de los distintos lienzo que generemos para luego su paso al backend y posterior renderización. Por defecto, canvas puede guardar los datos en formato mapa de bits con la función de `getImageData()`. Se ha escogido la solución nativa del lenguaje.

5.1.4. Decisión final:

Debido a la conveniencia de poderse hacer directamente con las funciones que aporta el lenguaje, quitando tiempo de aprendizaje o salvando el tiempo de investigación en caso de error o incompatibilidad con las herramientas o

frameworks añadidos, se tratará directamente la información que contiene los elementos canvas de html para el almacenamiento de información de un lienzo.

5.2. Sistemas de almacenamiento del lienzo e imágenes

Una vez obtenida la información añadida al lienzo necesitamos poder pasarlo a imagen, ya sea antes o después del paso de frontend a backend dependerá del método de envío sin embargo se debe plantear el formato de renderizado de las imágenes que condicionará el grado de pérdida o peso de la información a manejar por el resto de funcionalidades del proyecto.

5.2.1. Formatos de conversión

Hay muchos más formatos de guardado de imágenes de los que se estudiarán a continuación sin embargo se decide hacer una investigación reducida a los formatos más populares o que tengan mayor compatibilidad con los navegadores actuales, *JavaScript* tiene múltiples herramientas de conversión de formato sin embargo a la larga si usamos una extensión no compatible con la mayoría de usuarios la carga de trabajo aumenta pudiendo simplemente reducir costes teniendo la imagen en un único formato leible tanto en navegador, como archivo y con cualquier visor de imágenes incluido en una máquina estándar.

5.2.1.1. JPEG

Joint Photographic Experts Group es el comité de expertos que creó un estándar de compresión y codificación de archivos e imágenes fijas con el mismo nombre [15], es el formato original de guardado de imágenes estandarizado, debido a esto confiamos en su compatibilidad con todos los navegadores independientemente del hardware o software, sin embargo podemos ver mejores opciones en cuanto a compresión como vimos en la tabla de comparación entre jpeg y su sucesor más inmediato 3.1.3, dando lugar a la experimentación con otros formatos.

5.2.1.2. AVIF

El formato de archivo de imagen *AV1* o *AVIF* es una especificación de formato de archivo de imagen para almacenar imágenes o secuencias de imágenes comprimidas *AV1* en el formato de contenedor *HEIF* (High Efficiency Image File Format). Este es la extensión más reciente a estudiar, presenta un grado de compresión mejor que *JPEG* con menos errores gráficos de manchas de color o problemas con los bordes dando lugar a un formato sólido de almacenamiento de imágenes; sin embargo su reciente lanzamiento choca con las herramientas de renderización de video para las animaciones dando lugar a un formato

que estaría mejor orientado a documentos gráficos estáticos como fotografías o imágenes publicitarias.

5.2.1.3. GIF

Graphics Interchange Format es un formato de archivo rasterizado diseñado para imágenes relativamente simples generalmente para navegadores y entornos web. Este caso es distinto al resto dado que permite pequeñas animaciones directamente incorporadas en un archivo unitario a cambio de sacrificar grado de calidad y número total de colores disponibles, se plantea su uso para acortar pasos en el paso a video dado que tendríamos directamente el resultado visible. Sin embargo se descartará el uso en este proyecto debido a sus reglas en cuanto a sucesión de imágenes así como su reproducción sin opciones para detener o dejar de reproducir en bucle que nos ofrece un formato de video estándar.

5.2.1.4. PNG

Portable Network Graphics es un tipo de archivo para imágenes basado en un algoritmo de compresión sin pérdida para mapas de bits. Este formato fue desarrollado en buena parte para solventar las deficiencias del formato GIF y permite almacenar imágenes con una mayor profundidad de contraste y otros datos importantes como transparencia. Gracias al algoritmo *DEFLATE* o deflación, permite de manera similar a un archivo comprimido zip tener una compresión sin pérdidas debido al uso de la información de la cabecera creada.

5.2.1.5. Webp

Es un formato orientado en casi su mayoría a su manejo online, es ejecutado por el navegador en la mayoría de casos y puede presentar tanto guardados con pérdida como sin pérdida de información; además cuenta con *WebM*, un formato contenedor multimedia abierto y libre desarrollado por Google y orientado para usarse con HTML5. El único problema que presenta este formato es que ciertos navegadores no soportan su formato de imagen y video y sobretodo los programas no especializados de tratamiento multimedia no tienen opción para *Webp* complicando trabajar con este formato.

5.2.2. Formato escogido

Estudiando las posibilidades analizadas necesitamos buscar un formato que pueda ser manejado por cualquier tipo de navegador estándar, salvando muchos problemas de conversión o reinterpretación de datos que puede dar lugar a pérdidas de información importante para el usuario o para conservar la idea original del dibujo trazado. Estas imágenes que se crean no son como fotografías de

un espacio o elemento del mundo real que tiene que ser adaptado a un esquema de colores y una resolución dada por un dispositivo concreto, las imágenes que tratamos son datos directamente introducidas en un mapa de bits como si creáramos directamente una serie de variables con información ya dada, estas imágenes rasterizadas son más similares a un vector de elementos que a una fotografía real. Sabiendo esto deberíamos escoger un formato que además nos permita cierto grado de compresión sin pérdida dado que facilitaría la manutención de la base de datos y alargaría su vida útil economizando así su espacio total ocupado.

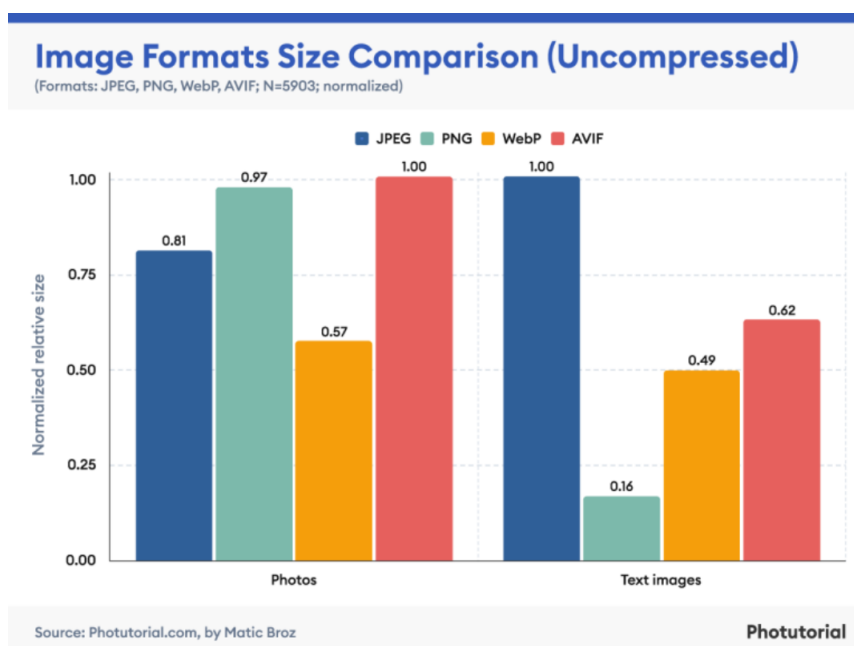


Figura 5.3: Imagen de lienzo con trazo de mapa de bits [4]

Después del periodo de experimentación con formatos se ha decidido optar por el formato **PNG**, debido a su compatibilidad con cualquier visor de imágenes y navegador moderno lo hace una herramienta sumamente útil de usar; como vemos en la gráfica adjunta posee la mejor compresión a la hora de tratar con imágenes de texto (similares a las imágenes rasterizadas que se generan) y generalmente no presenta errores de corrección de colores dentro del esquema ofrecido por el lienzo de *JavaScript* haciéndolo el formato ideal para generar la salida del elementos Canvas.

5.3. Bases de la animación y creación de video

Respecto a la creación de video a partir de fotogramas o imágenes sueltas en nuestro caso no existen tantas opciones como uno puede esperar en un len-

guaje tan extendido como *JavaScript*. Si el proyecto hubiera sido planteado en *python* se habrían disponido de una infinidad de librerías que permiten esto; que no haya una gran cantidad disponible no desmerece las que sí están presentes empleando la que puede ser la más popular para estos casos en desarrollo web, *FFmpeg*. Es un framework popular de manejo multimedia actualizado con rigurosa regularidad (según se escribe este apartado la última actualización fue la 8.0 el 22/08/2025) con capacidad de reproducción, edición, codificación y decodificación que nos permitirá la renderización de un video con los fotogramas creados localizados en una carpeta donde accederá nuestra parte del backend.

FFmpeg como hemos visto comprende una enorme cantidad de utilidades, comprendiendo una librería de múltiples herramientas para la edición de video y audio, sin embargo nos interesa principalmente su función de renderizado o codificación de video. Llamamos codificación de video a la creación de un archivo de reproducción de imágenes a partir de datos *RAW* o en este caso imágenes individuales que conforman los fotogramas del vídeo, teniendo la capacidad de espaciarlos dado un tiempo dado, aplicarles un filtro general a todos para uniformar la salida, regular la aceleración de la animación (un concepto de animación tradicional que afecta a la forma de espaciar en el tiempo los frames respecto según avanza para dar una sensación de velocidad específica) y más opciones a introducir en la línea de codificación de video a la que implementamos en *Nodejs* con la llamada `ffmpeg().videoCodec('tipo de codificador')` [29].

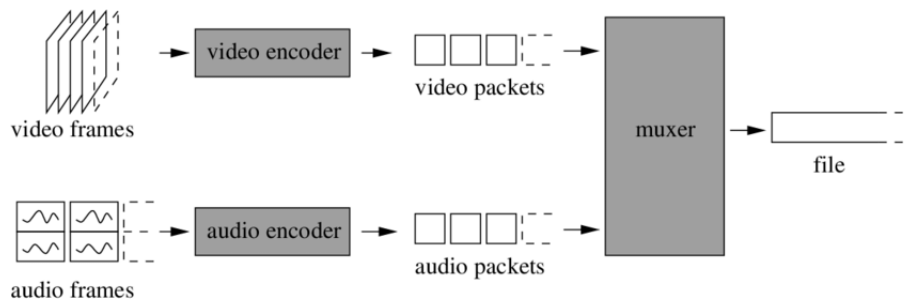


Figura 5.4: Proceso de codificación de *FFmpeg* [29]

Dispone de múltiples opciones de codificación que varía en opciones de display y transformación de la escala cromática de los frames originales, habiendo analizado que solo vamos a tratar con imágenes rasterizadas o mapas de bits la opción más recomendada por los usuarios es el codificador *libx264* o *libx264rgb*.

Este codificador, además de ser el más documentado, tiene el mayor número de opciones a la hora de analizar el producto resultante pudiendo ver el porcentaje de creación, espacio de tiempo entre fotogramas y demás opciones que pueden mejorar la accesibilidad y opciones de la aplicación en general. Presenta también de dos variantes según el formato de colores a emplear, sea *YUV*, el espacio de color usado habitualmente como parte de un sistema de procesa-

miento de imagen en color o el más clásico siendo RGB; vamos a emplear la primera debido a la correspondencia con los colores ofrecidos por el *color-picker* de *JavaScript*.

El estándar de codificación de *Libx264* es una norma que define un códec de vídeo de alta compresión, desarrollada conjuntamente por el *ITU-T Video Coding Experts Group (VCEG)* y el *ISO/IEC Moving Picture Experts Group (MPEG)*; capaz de proporcionar una buena calidad de imagen con tasas binarias notablemente inferiores a los estándares previos (MPEG-2, H.263 o MPEG-4 parte 2), además de no incrementar la complejidad de su diseño [24]. Esta nueva norma surgió de la necesidad de conseguir un muestreo de 4:4:4 o 4:2:2, funciones para la mezcla de escenas, tasas binarias más elevadas y poder representar algunas partes de video sin pérdidas así como utilizar el sistema de color por componentes RGB. Por este motivo surgió la necesidad de programar unas extensiones que soportasen esta demanda. Tras un año de trabajo intenso surgieron las "extensiones de gama de fidelidad" (FRExt) que incluían:

- Sustento de un tamaño de transformada adaptativo.
- Sustento de una cuantificación con matrices escaladas.
- Sustento de una representación eficiente sin pérdidas de regiones específicas.

Este conjunto de extensiones denominadas de "perfil alto" son:

- La extensión High que admite 4:2:0 hasta 8 bits/muestra.
- La extensión High-10 que admite 4:2:0 hasta 10 bits/muestra.
- La extensión High 4:2:2 que admite hasta 4:2:2 y 10 bits/muestra.
- La extensión High 4:4:4 que admite hasta 4:4:4 y 12 bits/muestra y la codificación de regiones sin pérdidas.

Estas extensiones son usables por FFmpeg cada una tiene sus propias características que compensan la implementación de una u otra según la especificación que se requiera:

Especificación	Original	High	High 10	High 4:2:2	High 4:4:4
slices I y P	SI	SI	SI	SI	SI
slices B	NO	SI	SI	SI	SI
slices SI y SP	NO	NO	NO	NO	NO
filtro "deblocking"	SI	SI	SI	SI	SI
codificación CAVLC	SI	SI	SI	SI	SI
codificación CABAC	NO	SI	SI	SI	SI
ordenación flexible de macrobloques (FMO)	SI	NO	NO	NO	NO
formato monocromo (4:0:0)	NO	SI	SI	SI	SI
formato 4:2:0	SI	SI	SI	SI	SI
formato 4:2:2	NO	NO	NO	SI	SI
formato 4:4:4	NO	NO	NO	NO	SI
8 Bits/píxel	SI	SI	SI	SI	SI
9 y 10 Bits/píxel	NO	NO	SI	SI	SI
11 y 12 Bits/píxel	NO	NO	NO	NO	SI
transformada 8x8	NO	SI	SI	SI	SI
matrices de cuantificación	NO	SI	SI	SI	SI
codificación sin pérdidas	NO	NO	NO	NO	SI

Tabla 5.1: Extensiones de H.264 [28]

La mayor ventaja que presenta *H.264* es la menor cuantía de información que se necesita almacenar en los videos codificados mediante este códec. Si

atendemos a estas especificaciones por orden vemos por ejemplo este estándar permite:

- Enviar imágenes intra (**imágenes I** o que se codifican por sí mismas) muy costosas en tiempos de procesamiento, pasar de un vídeo a otro utilizando predicción temporal o espacial como antes, con la ventaja que permiten la reconstrucción de valores específicos exactos de la muestra aunque se utilicen imágenes de referencia diferentes o un número diferente de imágenes de referencia en el proceso de predicción.
- Respecto al caso de *deblocking* integra un filtro **antibloques** que mejora la eficacia de compresión y la calidad visual de las secuencias de vídeo eliminando efectos indeseables de la codificación como por ejemplo el efecto de bloques.
- Para evitar pérdidas a la hora de compresión o envío de información, emplea la codificación entrópica, esta se puede realizar de tres formas diferentes. Un primer método utilizado es el conocido *UVLC (Universal Variable Length Coding)*. Este tipo de codificación es utilizado para codificar la gran mayoría de los elementos de sincronización y cabeceras. Los otros dos métodos son utilizados para codificar buena parte del resto de elementos sintácticos (coeficientes, vectores de movimiento). Las codificaciones utilizadas para esta tarea están basadas en *VLC (Variable Length Coding)* de forma adaptativa, de este concepto nace el **CAVLC (Context Adaptive Variable Length Coding)** y el **CABAC (Context Adaptive Binary Arithmetic Coding)**.
- En cuanto a la teoría, emplea la una aproximación a la *DCT (transformada discreta del coseno)* pero con ciertas características específicas:
 - El tamaño de la matriz 4x4 píxeles (8x8 en los perfiles FExt).
 - Empleo de coeficientes enteros para evitar los errores de redondeo habituales en la *DCT clásica (coeficientes irracionales)* y garantizar un ajuste perfecto entre la transformación directa y la inversa.
 - Posibilidad de calcular sin exceder los 16 bits de precisión.
 - Y puede implementar exclusivamente por medio de sumas y desplazamientos binarios.
- La cuantificación de imagen (*QP*) incrementa un 12,5% el intervalo de cuantificación, lo que equivale a duplicarlo por cada 6 pasos. El rango dinámico del *QP* ha aumentado respecto a normas precedentes, puesto que los valores van de 0 a 51. Los macrobloques se cuantifican utilizando un parámetro de control que puede cambiar adaptándose al bloque por cada bit adicional (partiendo de 8 bits, 52 pasos). Además, para poder conseguir los mejores resultados visuales la cuantificación de la crominancia es más esmerada que la de luminancia.

- Se ve enormemente beneficiado por el uso de distintos algoritmos de prevención de pérdidas
 - **FMO y ASO:** La *ordenación flexible de macrobloques* y la *ordenación arbitraria de slices* son técnicas para reestructurar la representación de las regiones fundamentales (macrobloques).
 - **DP:** La *partición de datos* proporciona la capacidad de separar los elementos de sintaxis más importantes de los menos importantes en paquetes de datos diferentes, permitiendo el uso de *protección de error desigual (UEP)*.
 - **RS:** El algoritmo de *slices redundantes* permite a un codificador enviar una representación suplementaria de una región de imagen que puede ser usada si la representación primaria es corrompida o perdida.

Para emplear este codificador en la librería *FFmpeg* solo tenemos que especificarlo en la opción de `videoCodec('libx264')`.

5.4. Diagrama de casos de uso

Para representar el modo de uso de este proyecto empleamos el diagrama de caso de uso donde mostramos la ejecución de el software desde el punto de vista de dos figuras que son los posibles clientes, una figura de usuario normal y de uno administrador, pudiendo realizar distintas operaciones sobre el conjunto de elementos del proyecto.

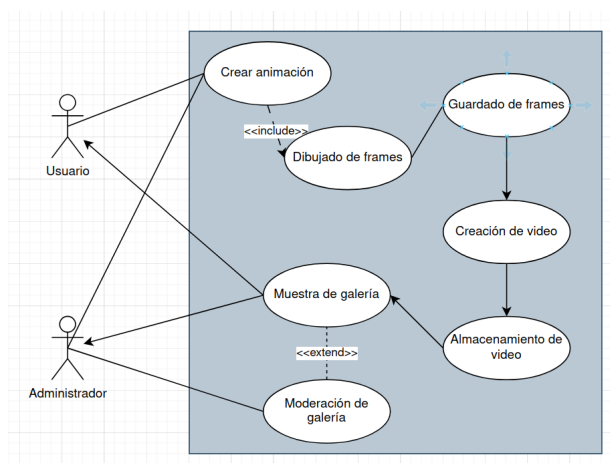


Figura 5.5: Diagrama de casos de uso de Proyecto HopBlog

Capítulo 6

Implementación

Después de una ardua investigación, se empieza a desarrollar lo que sería el código del proyecto, como se ha comentado antes el desarrollo ha sido llevado mediante una lista de tareas a cumplir repartidas en sprints de tiempo definido para poco a poco aumentar las funcionalidades y capacidad del prototipo. Todo el código ha sido desarrollado en *VisualStudio Code* junto al terminal de Linux para regular la actividad de la base de datos correspondiente al proyecto; el proyecto puede ser revisado y descargado en su repositorio correspondiente [2].

La estructura del repositorio se presenta de esta manera:

- **docs** : carpeta para memoria y archivo *.gitignore* .
- **public** : contiene el archivo de formato de estilo para las páginas renderizadas a los clientes, así como las funciones que usan. Además junto a las imágenes y videos subproductos del uso de la aplicación.
- **test**: localización de los tests de código.
- **views**: aquí colocamos las páginas a renderizar al cliente.
- *server.js* levanta el servidor en *NodeJS*.
- *funciones_sql.js* se trata del archivo específico para las funciones de manipulación de la base de datos.
- *video.js* es el archivo para funciones de renderizado y filtrado de videos.
- *package.json* son las especificaciones del proyecto para su ejecución en máquina local.
- **LICENSE**: la licencia de uso del programa acorde con *Free Software Foundation, Inc.* para su distribución y uso con crédito dado.
- **README.md**: archivo de instrucciones de uso del software.

6.1. Implementacion del sprint 1

6.1.1. Despliegue de NodeJS

Este primer sprint sirve para desplegar la base de lo que sería nuestro servidor donde residen las páginas a consultar, pudiendo ser desplegado desde una máquina de forma paralela y recibir consultas de clientes ajenos a esta, todo con un soporte web alojado en una ip específica. Como ya hemos elaborado, se emplea *NodeJS* para esta parte, es un entorno web que requiere instalación mínima y opera en lenguaje *JavaScript*; al desplegar una aplicación *NodeJS* con el comando `npm init -y` se crean automáticamente los archivos de *package.json* y *package-lock.json*, donde quedarán registradas las dependencias que instalemos en nuestro proyecto.

Junto a *NodeJS* instalaremos el framework de *Express* para ofrecer una forma rápida y sencilla de escribir el código del servidor, *Express* permite tener los parámetros como la dirección ip, puerto y las páginas dentro de esta dirección localizadas de forma sencilla y sus funciones modularizadas para saber a qué corresponde cada salida de la dirección relativa que pongamos en el navegador.

Por último, por comodidad a la hora del desarrollo y de las pruebas manuales, instalamos *nodemon*, un *Daemon* que permite ejecutar el servidor de *NodeJS* de fondo y se actualiza cada vez que guardemos un cambio en el código del script del servidor lanzado.

Ahora solo necesitamos establecer la dirección donde vayamos a ejecutar el servidor, por ejemplo he tomado **127.0.0.1** y el puerto **3000** para desplegar el servidor, declarando así el servidor y sus parámetros de despliegue.

6.1.2. Creación del lienzo

Para crear lo que corresponderá a nuestro lienzo de dibujo del proyecto necesitamos una página html donde irán los elementos correspondientes en este caso un elemento *Canvas* con una id específica a poder ser dentro del cuerpo de la página html, dada esta página necesitamos asignarle las funciones que aportará la lógica a los elementos creados por el navegador así que enlazándole el script de *JavaScript* de **funcionescanvas.js** podemos empezar a escribir el código detrás de las funciones de interacción de canvas.

Podemos acceder a este a través del script gracias a

```
document.getElementById('canvas');
```

que busca el elemento con id canvas en la página html asociada, necesitamos ciertos parámetros como el tamaño de ancho y largo para hacer la diferencia con respecto al tamaño de la ventana a la hora de localizar el cursor y el una variable que aloje el contenido actual del lienzo. Teniendo esto debemos crear eventos para los estados del ratón de:

- touchstart con función de start();

- touchmove con función de draw();
- mousedown con función de start();
- mousemove con función de draw();
- touchend con función de stop();
- mouseup con función de stop();
- mouseout con función de stop();

El funcionamiento es sencillo, para la función `start()` toma el inicio del trazo como el punto donde se ha clickado dentro del lienzo, a continuación la función de dibujar consiste en continuar la trayectoria del ratón en la posición relativa dentro del lienzo uniendo los píxeles clickados coloreando esa parte seleccionada, aquí se declara el grosor del trazo y su color; finalmente en los casos de salirse del lienzo o dejar de clickar se deja de colorear la trayectoria y se fija en el lienzo interactuado. Hay que asegurarse de que deje de registrar el movimiento del cursor una vez fuera del lienzo para no activar involuntariamente la función de dibujar al volver a entrar al lienzo, esto fue solventado según se fue implementando.

6.1.3. Guardado del contenido del canvas

Como hemos discutido anteriormente se puede acceder directamente al contenido del elemento *Canvas* si buscamos el elemento con el id específico dentro del archivo html y una variable que contenga el contenido del lienzo

```
let context = canvas.getContext('2d')
```

A continuación creamos un vector que contenga los distintos *context* a lo largo de la secuencia de fotogramas, teniendo así un array con los distintos contenidos del lienzo. Ahora accedemos a los contenidos de este array con la función

```
context.getImageData(0, 0, canvas.width, canvas.height)
```

Así podemos recuperar el estado de un lienzo anterior antes de cambiarlo o borrarlo siempre que haya sido almacenado en nuestro array de fotogramas, en esto se basarán las funciones posteriores de guardado y envío de fotogramas.

Inicialmente se empezó probando la conversión directa del lienzo a *PNG* para descarga directa y testeo manual del sistema; recurriamos al sistema de conversión de contenido del lienzo a tipo de archivo *Blob* para guardarse directamente en hardware con extensión *PNG*. Esto es un método temporal, el objetivo no es que el cliente descargue los fotogramas directamente en su máquina.

6.1.4. Herramientas del lienzo

La inspiración principal de este proyecto es el software de *Flipnote Studio*, una herramienta de dibujo que ofrece múltiples herramientas de ilustración para

acelerar o añadir nuevas experiencias al dibujo, este programa cuenta con una gran cantidad de herramientas las cuales se han seleccionado unas cuantas para luego posterior implementación de estas o revisión de la aplicación en caso de que se vea necesaria su implementación, se ha decidido escoger estas:

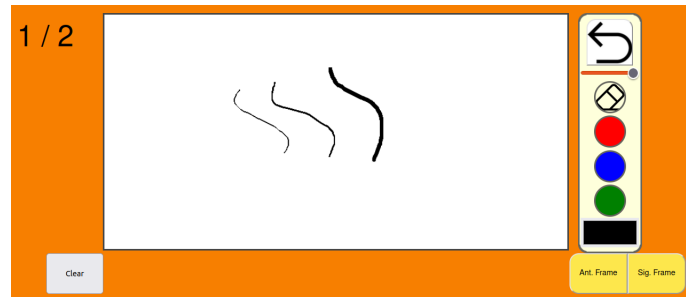


Figura 6.1: Lienzo del proyecto

6.1.4.1. Contador de frames

Para tener constancia del número de frames o por cuál vamos tenemos un cuadro de texto que nos indica la posición actual en el array de frames totales y la cantidad total respectivamente.

6.1.4.2. Avance o retroceso de frames

Dos botones que indican el paso a un frame u otro, se basan en el uso de array de almacenamiento de contenido del lienzo, teniendo las características de

- Aumentar el tamaño del array de frames si se avanza más del número total de frames actual.
- El efecto de crear un nuevo frame, que consiste en guardar el estado actual del lienzo, borrar el contenido de este y reservar un espacio en el array para este nuevo frame.
- Detener en caso de retroceder por encima del primer frame.
- Cargar el frame correspondiente si este ha sido dibujado.

Todo se basa en un sistema de carga y descarga del array de frames y el contenido del elemento *Canvas*.

6.1.4.3. Limpiar lienzo

Es una función realmente sencilla simplemente vaciando el contenido del frame actual dentro del array de frames y dejando solo el color del fondo de lienzo como resultado, vaciando efectivamente el lienzo.

6.1.4.4. Cambio de tamaño del trazo

Es un botón tipo slider, es decir un botón que se desliza sobre una guía, de 3 valores; el por qué solo 3 valores de grosor es debido a que *Flipnote Studio* empleaba únicamente 3 valores de grosor también simplificando el sistema y siendo más adecuado para un sistema de dibujo que es más aparente a un conjunto de pixels que a por ejemplo un dibujo tradicional a lápiz. Este slider tiene de máximo 1 al 3, el grosor de 3 ha sido aumentado con respecto al resto para mayor versatilidad como el programa original.

6.1.4.5. Cambio de color

El color base de escritura es negro, sin embargo para añadir variedad se ha añadido tres colores rojo, azul y verde, además de un color picker pudiendo escoger todos los colores que ofrece html para gusto del autor. El sistema de muy simple sencillamente volviendo a declarar el color del trazo a dibujar.

6.1.4.6. Goma de borrar

Una goma de borrar no necesita realmente borrar el contenido como tal del lienzo, simplemente dibuja con el color del lienzo de fondo simulando así habiendo borrado esa parte del dibujo.

6.1.4.7. Deshacer acción

Consiste en eliminar el último movimiento realizado en el lienzo para arreglar errores a la hora de dibujar, este método consiste en mirar dentro del array de frames, en un frame en concreto registrar todos los movimientos realizados por el lienzo y eliminar el último elemento del array de arrays de los frames total y cargara sobre el lienzo; es una operación sencilla debido a la ventaja de ser un sistema de mapa de bits.

Este primer sprint inicial estaba calculado para 2 semanas, aproximadamente abarcando el espacio de tiempo de 14/3-30/3 con sesiones de entre 2 a 3 horas hasta llegar al producto mínimo funcional para probar el funcionamiento e investigación previa a este, aunque esté sujeto a pruebas las bases del proyecto han sido fundadas para poder seguir avanzando.

6.2. Implementacion del sprint 2

Este sprint se centra en el funcionamiento del sistema servidor-cliente, probando el paso de información entre distintos lados de la aplicación así como la mejor forma de pasar la información que contiene el lienzo para luego procesarla en servidor, el objetivo es que el cliente no tenga que emplear su hardware para procesar la animación.

6.2.1. Paso de frontend a backend

Tenemos un botón en la zona del lienzo que llama a las funciones de paso de información al servidor, estas funciones en el script de *JavaScript* se encargan de extraer el array de frames totales actuales y pasarlo a la parte del servidor para operar con ellos. Se dedicó bastante tiempo a cómo sería la manera ideal de pasar una imagen rasterizada para su almacenamiento y postprocesado además de una buena forma de enviarse por red sin alterar su contenido, *JavaScript* tiene funciones de envío con información con la llamada a `fetch()` pudiendo hacer una petición, sin tener que cargar la página correspondiente, a una dirección específica adjuntando la información correspondiente que espera recibir.

El proceso sería el siguiente: desde el lado del cliente con las funciones de *JavaScript* hacemos una petición a la página correspondiente que reside en nuestro servidor; esta dirección llama dentro del servidor a una petición tipo **POST** (es decir de transferencia de datos) donde recibirá el cuerpo de la petición efectuada por `fetch`. Esta petición `fetch` envía datos tipos *JSON*, es decir necesitamos hacer de los elementos del array de frames codificarlos para pasarlos como un mensaje de caracteres *ASCII*.

En el cuerpo de la petición enviamos el contenido del lienzo codificado, además su posición dentro del array de frames para cuando generemos de vuelta la imagen conserve su posición y tenga sentido el orden de los fotogramas a la hora de generar el vídeo.

En el envío a la dirección también incluimos los parámetros de un identificador numérico de la animación conjunta, es decir un número de 5 cifras que identifica a los frames que conforman una misma animación. Además también tenemos de parámetro el nombre puesto por el cliente de la animación que incluimos en el formulario de envío de la animación, nosotros identificaremos los frames de una animación por el id generado asegurando así que el usuario pueda poner el nombre que quiera aunque ya haya sido tomado por otro usuario u otra animación.

Habiendo enviado ya mediante `fetch` cada frame creado, en la parte del servidor creada para recibir esta información necesitamos declarar que haga una excepción en el sistema de *CORS* (*Cross-Origin Resource Sharing*), esto es un mecanismo de seguridad implementado por los navegadores web que permite restringir las peticiones HTTP realizadas desde un dominio a otro diferente que se tiene por defecto, así que declaramos en el servidor de *Node* que no intervenga en esta petición y declaramos una cabecera de permiso para las respuestas con operaciones de envío de datos

```
res.setHeader('Access-Control-Allow-Origin', origin) .
```

6.2.1.1. Creación de base de datos

Mediante *MySQL* vamos a emplear un servicio de peticiones con las base de datos que nos servirá para almacenar la información de los clientes y usuarios

que pertenecen a la plataforma.

Se instaló *MySQL* para ir creando la base de datos con las tablas correspondientes a los distintos tipos de información a guardar, para este trabajo se han asignado un host y un usuario nuevo para modularizar el trabajo. Todos estos valores deben declararse como variables de entorno para el proyecto, es decir son parámetros que usaremos para desarrollo de software sin estar dentro del código base, es un archivo oculto denominado *.env*.

Para ejecutar funciones sql en *NodeJS* se incluyó la librería *mysql2*, que actúa como un analizador sintáctico de *MySQL* nativo, para conectar con la base de datos creada necesitamos declarar las variables de entorno mencionadas con la librería *dotenv*, un sencillo módulo para cargarlas. Ejecutamos la conexión a esta base de datos lanzando un operación asíncrona de promesa. Esta separación entre variables y conexión añade una capa de seguridad y ordenación al código para evitar cualquier modificación ajena que pueda ser dañina así como la jerarquización de los datos que empleamos en el código.

```
export async function getFramesById(id) {
  const [rows] = await pool.query(
    `SELECT *
     FROM frames2
     WHERE id = ?
     `, [id])
  return rows
}
```

Con todo esto podemos empezar a crear sentencias sql para modificar la base de datos y exportarlas según se necesiten en funciones del servidor concretas, con el detalle de comprobar previamente si las variables introducidas no son algún intento de inyección de código sql gracias a este formato de escritura.

6.2.1.2. Almacenamiento del lienzo

Una vez creada tanto como el sistema de envío del front-end al back-end y el acceso a la base de datos desde nuestro código, podemos empezar a introducir los frames que vayamos recibiendo en nuestra base de datos personal.

Habiendo recibido la información del número de frame, identificación de la animación, nombre de la animación y contenido del lienzo podemos hacer una entrada en nuestra tabla de frames. Esta tabla contiene 4 parámetros:

- Id de frame: un número aleatorio de 6 cifras que identifica a un frame de forma unívoca. Se guarda como valor entero.
- Id de animación: el identificador de 5 cifras que pasamos en la url para identificar el video resultante. Se guarda como valor entero.
- Título: es el título o nombre de la animación que pone el usuario, este se pasa por un filtro para quitar los espacios y se sustituye por guión bajo. Tipo de dato *VARCHAR(30)* es decir no más de 30 caracteres.
- Frame: es el contenido del frame en caracteres *ASCII* para su almacenamiento en tipo de dato *TEXT*, que permite un tamaño de hasta 64 KB.

Finalmente se añade todo, añadiendo a la información del frame *base64* para luego generar la imagen correspondiente.

El segundo sprint se le destinó 6 semanas, fechas aproximadas de entre el 12/4 al 24/5 con bastantes más periodos de prueba debido a la dificultad de paso de front-end a back-end sin alterar demasiado el cuerpo de la petición de envío; pudiendo finalmente mediante el sistema de envío con información codificada por *JSON* junto a los datos pertinentes para almacenamiento de estos en su base de datos correspondiente.

6.3. Implementacion del sprint 3

A continuación teniendo los frames almacenados y codificados podemos empezar a crear las animaciones que corresponden al envío original de estas; al igual que la función de envío anterior necesitamos tener activado el *Access-Control-Allow-Origin* debido a las normas de *CORS* de los navegadores.

6.3.1. Renderización de imágenes

El proceso de guardado de frames está enlazado al de renderización debido a que el botón que pulsamos como botón es el de creación de animación. Esto quiere decir que al pulsar el botón desde el archivo html llamamos a 3 funciones: guardar lienzos que forman los fotogramas, renderizar imágenes para fotogramas y renderizar video. Las dos últimas operaciones se hacen haciendo una petición a una misma dirección debido a que necesitamos tener almacenadas de forma física los fotogramas como imágenes *PNG* en una carpeta para acceder desde el servidor.

Tenemos una tabla de la base de datos dedicada a los fotogramas que conforman las animaciones de los usuarios, buscamos los fotogramas en la tabla de fotogramas con la id de animación correspondiente al video de salida. La consulta dará lugar a varios resultados por tanto los recorreremos con un bucle y separamos la información sobre el número de frame correspondiente a este y la información relativa al contenido de la imagen; con el contenido ahora necesitamos guardarlo como una imagen formato *PNG*, mediante el módulo de *File System* podemos convertir el valor *ASCII* codificado en archivo *base 64* y guardarlo en un archivo formato *PNG*, haciendo esto por cada fotograma en una carpeta del servidor.

6.3.2. Renderización de video

Teniendo los fotogramas de video en una carpeta, podemos empezar a renderizar el video que tenía como objetivo el cliente. Se empleará el framework de tratamiento de archivos multimedia de *FFmpeg* con el estándar de codificación de *h.264*.

Habiendo cargado todos los fotogramas en la carpeta de almacenamiento simplemente esta función toma todos los archivos que tengan de nombre *frame%01d.png* y los guarda en la carpeta de videos totales donde se le pondrá de título el id de la animación conjunta. La carpeta de fotogramas puede estar llena debido a su uso en la anterior animación así que se vacía para asegurarse de que los fotogramas son los nuevos. Finalmente queda registrado en la base de datos en la tabla de videos con su correspondiente id, autor y dirección del vídeo.

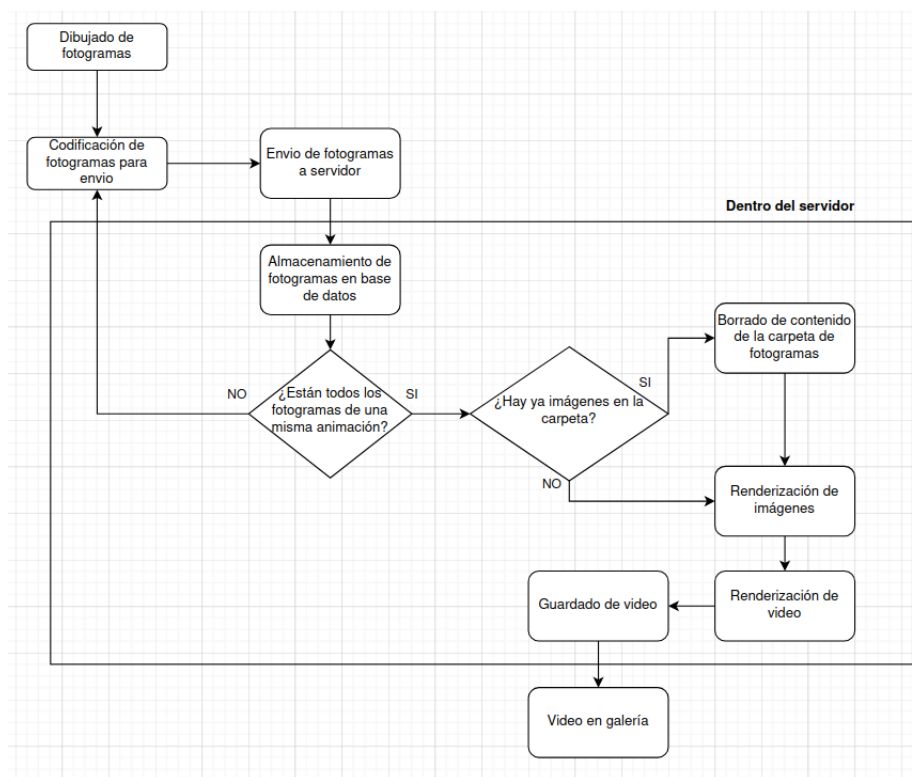


Figura 6.2: Diagrama de flujo del proceso de guardado de un video desde que se dibuja hasta que se envía al servidor

Este sprint es la base del *producto mínimo viable (MVP)*, nos permite crear animaciones y dibujar de forma libre en un navegador cumpliendo con un objetivo básico del proyecto y además tachando todas las tareas de prioridad alta de la lista. Este sprint se planeó para 3 semanas y abarca aproximadamente desde el 24/5 hasta la semana del 14/6.

6.4. Implementacion del sprint 4

Este último sprint se ha empleado en la implementación de sistemas para el entorno web así como comprobación de las medidas de seguridad o posibles fallos que pueda ser sometido el proyecto entero. Esto añade a la experiencia del portal web pudiendo satisfacer al cliente con una mejor experiencia de uso así como para cualquiera que aporte al código como integrante del equipo o interesado en nuevas funciones para el software.

6.4.1. Sistema de usuarios

Dado que este proyecto sirve para ofrecer un portal web para gente con objetivos tanto lúdicos como prácticos se decidió añadir un sistema de autoría y usuarios para saber quién forma parte de la comunidad. Se establece la regla de que para crear una animación se necesita de autoría para siempre tener constancia de quién realiza qué animación o avisar en caso de infringir las normas de comportamiento.

Para crear un sistema de login y usuarios en *NodeJS* se incluye en *Express* las variables de encriptación de sesión. Necesitamos indicar qué conforma una sesión, para ello debemos escribir las opciones de sesión en las que mínimo tenemos que especificar:

- **Secret:** palabra clave o cifra cualquiera usada para firma la cookie de ID de sesión, puede ser cualquier tipo de valor soportado por *NodeJS crypto.createHmac* puede ser tanto un una única clave como un array de claves distintas; se aconseja que sea un valor aleatorio no introducido por un humano.
- **Resave:** fuerza que la sesión sea guardada en el almacén de sesiones incluso si ha sido modificada durante la petición de inicio. Si se tiene la función *touch* de *Express* se iguala a *false*, por defecto se encuentra activada.
- **saveUninitialized:** fuerza que la sesión sea sin iniciar para guardarla en el almacen, es decir cuando es nueva sin ser modificada. Esto reduce la carga de peticiones de la página.

```
app.use(session({
  secret : '12345689',
  resave : false,
  saveUninitialized : false
}));
```

Habiendo establecido estos parámetros de sesión podemos registrar usuarios gracias al uso de base de datos; añadiendo así otra capa de utilidad añadiendo un sistema de usuarios clientes normales y administradores con capacidad de moderar aportes a la galería. En nuestro caso ponemos *false* a ambos debido a que cumplimos las características de *Express* para aprovechar la capacidad de almacenamiento de la sesión.

6.4.2. Sistema de privacidad y moderación

Para crear una comunidad sana equivalente a la que sostenía *Flipnote Studio* es necesario de regulaciones o normas generales para cualquier usuario nuevo o administrado metido en el proyecto, dando lugar a la implementación de seguridad dentro de la exploración de la herramienta web para asegurar el correcto desarrollo del proyecto y método de empleo.

6.4.2.1. Sistema de privacidad y búsqueda

Cuando se crea un vídeo, previamente se le asigna un nombre al archivo para titular la obra creada, entre eso y los valores implícitos en el envío un video dispone de: id particular de animación, autor y nombre. Para mejorar la experiencia se ha querido crear un sistema de etiquetas de videos y privacidad.

El sistema de etiquetas es una serie de palabras encadenadas por comas donde un usuario describe el contenido o los valores de la animación, es decir si por ejemplo se dibuja una cara el usuario puede poner en las etiquetas del video **cara** o cualquier otro parámetro que vea pertinente. Este sistema de etiquetas está condicionado única y exclusivamente por a comunidad, donde se toman temas comunes y se asignan para los contenidos de las animaciones.

Gracias a estas etiquetas podemos implementar un sistema de búsqueda, donde mediante envíos de formularios post-procesamos sus peticiones de búsqueda para filtrar los vídeos de la galería según los gustos del cliente, también podemos filtrar autores concretos para saber qué es atribuido a qué usuario dentro de la comunidad, creando así una identidad dentro del portal web y un aliciente para descubrir más animaciones o buscar recursos concretos dentro de la base de datos.

También se ha querido añadir un parámetro de privacidad escogible para gusto de usuario. Esta es una herramienta de aprendizaje y algunos clientes no se sienten confiados en compartir su trabajo, se da la opción de que solo sea visible por ellos mismos. Simplemente añadiendo un valor booleano a los vídeos dentro de la base de datos limitamos su visibilidad dentro de la galería, para gusto del cliente.

6.4.2.2. Jerarquía de usuarios

Queremos establecer una comunidad de usuarios donde se pueda compartir y crear los trabajos creados dentro de la plataforma dentro de unas normas de comportamiento establecidas. Este servicio web no tiene una edad límite para ser accedido sin embargo sería ideal establecer un control ante qué se sube o qué está permitido registrar en las base de datos públicas.

Dado que esto es un sistema de dibujo libre, no hay realmente patrones para reconocer por una máquina, así que se crea la figura del **administrador**; un miembro del equipo o elegido por ellos que se encarga de eliminar aquellos aportes a la página que pueda comprometer las reglas de comportamiento de la

web. para ello en la tabla correspondiente a los usuarios se crea el parámetro de rol, donde por defecto si se crea un usuario solo puedes ser usuario normal; se necesita de un miembro del equipo, es decir un administrador, para conseguir el rol.

Los administradores también puede animar y ver las galerías de usuarios específicos pudiendo así moderar de forma controlada a gente según sus post realizados. Cuentan con la capacidad de ver todas las animaciones de la galería incluyendo aquellas con el bit privado debido a su capacidad de moderación, siendo solo visibles por el autor y administradores de la página, sin comprometer los deseos de privacidad del cliente ante un público mayor para el acuerdo de privacidad y autoría consecuente.

6.4.3. Medidas de seguridad y prevención a la inyección de código

Debido a la cantidad de formularios, peticiones y en general envío de datos al servidor la información puede ser comprometida por aquellos que busquen el sabotaje de la información ajena dentro de la comunidad de *Proyecto HopBlog*. Se han impuesto ciertas medidas para el caso de vulneración de código:

- Todas las funciones de sentencias sql tienen un filtro para no modificar la base de datos directamente, gracias a la capa de abstracción de *Express* podemos comprobar si la petición a introducir como valor puede formar una *query* de sql o no.
- Múltiples páginas han sido bloqueadas solo para aquellos con usuario iniciado, en general aquellas que consistan en introducir información o peticiones al servidor. Devolviendo a la página principal en caso de falta de autenticación a la hora de animar o intentar moderar sin permiso.
- Redirecciones y bloqueos en caso de salto a distintas partes del proyecto escribiendo la dirección url, mediante comprobaciones en las bases de datos y avisos de redirección se evita el forzar la entrada a páginas sin permiso o rompiendo la secuencia de uso para su acceso.

Este último sprint, aunque mucho más ligero en cuanto a exigencia de funciones, se le estimó una duración de 3 semanas similar al anterior debido a la revisión de múltiples sistemas así como la buena implementación sin alterar el comportamiento de las funciones previas y añadir el apartado de seguridad y mejoras de la calidad del producto; aproximadamente de 1/7 al 14/7 se estuvo revisando el proyecto y las bases de datos empleadas.

6.5. Diagrama de clases

Se ha creado un diagrama de clases UML para describir el sistema de clases y funciones empleada en el código:

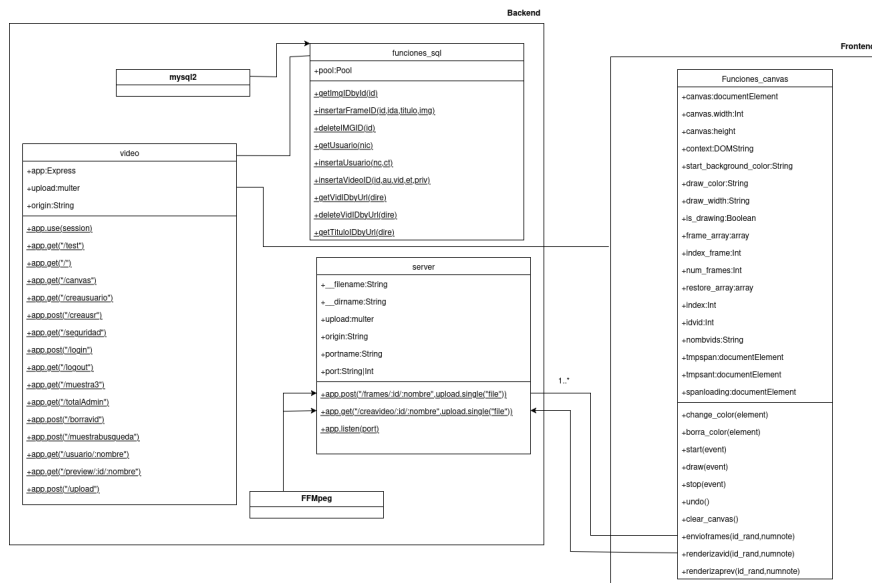


Figura 6.3: Diagrama de clases del proyecto

6.6. Diagrama de arquitectura

Aquí podemos entender el uso de la aplicación con el diagrama de arquitectura donde vemos que el código está alojado en la nube donde se pasan las peticiones del usuario entre los distintos módulos del proyecto hasta llegar a la información alojada en la base de datos de *Clever Cloud*

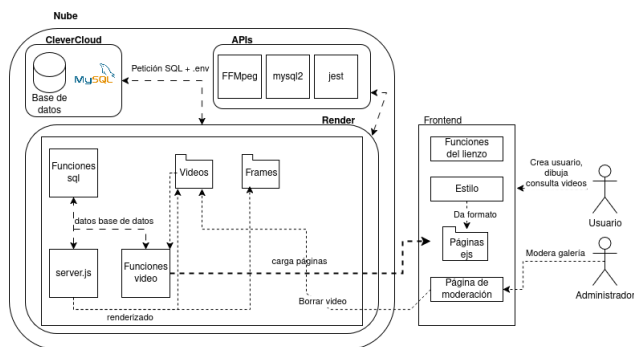


Figura 6.4: Diagrama de arquitectura

6.7. Diagrama de interfaces

Con este diagrama vemos un esquema del diseño de las páginas y el recorrido de estas, es decir cómo saltamos de una a otra. Todo esto está definido en la hoja de estilo *CSS* del proyecto:

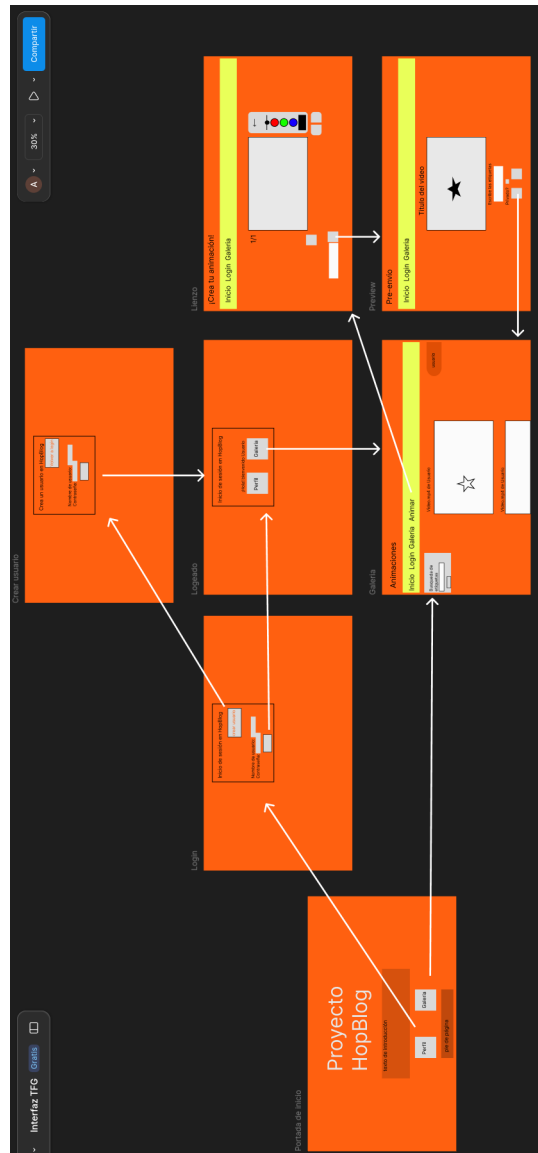


Figura 6.5: Diagrama de la interfaz de las páginas

6.8. Despliegue en red

Una vez probado todo el proyecto, sería conveniente estudiar sus resultados en un entorno web real, es decir desplegar el servidor como un servicio web con su propia dirección url para hacerlo verdaderamente sostenible de forma autónoma en la nube.

El proceso de implementación en la nube puede ser un proceso costoso a nivel computacional, lógico e incluso económico debido a las condiciones que se tienen que soportar para que una página sea accesible a través de red; máquinas especializadas en este proceso como son servidores remotos o incluso ordenadores dedicados exclusivamente a ello son empleados con el fin de mantener la información en todo momento posible para asegurar el servicio constante de la página ofrecida a los clientes. La accesibilidad de internet es su mayor virtud y su mayor desafío a partes iguales. Sin embargo hoy en día para proyectos pequeños se pueden tomar alternativas no tan exigentes y al alcance del desarrollador medio, tanto con planes de pago asequibles como directamente gratuitos, eso sí limitando el espacio de almacenamiento debido a las posibles pérdidas y mantenimiento ajeno del código.

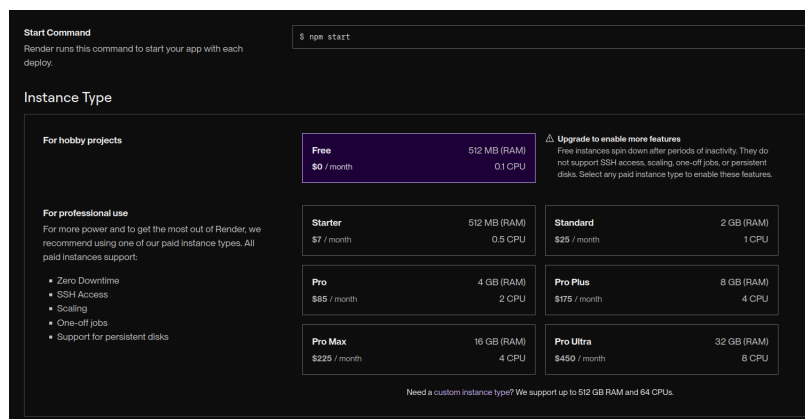


Figura 6.6: Plan de servicio de *render.com* [27]

6.8.1. Despliegue de Proyecto HopBlog

Para este proceso hemos empleado dos herramientas de desarrollo en la nube.

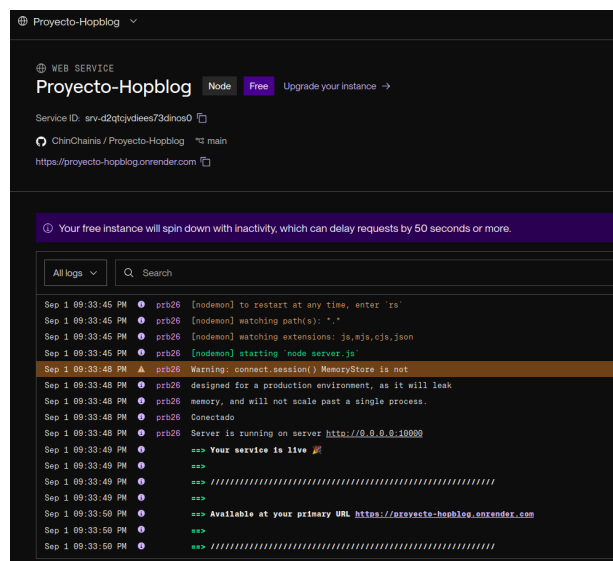
- *CleverCloud*: La plataforma de automatización de *TIC* (tecnologías de la información y comunicación) proveyendo de soluciones de *PaaS* (plataforma como servicio).
- *Render*: el servicio web donde podemos construir, desplegar y escalar aplicaciones de forma sencilla sin apenas instalación.

Hay que añadir que ambas se han empleado con el plan gratuito.

Para la implementación en *CleverCloud* solo se ha requerido de crear una cuenta y seleccionar el despliegue de una base de datos de *MySQL*. Pudiendo crear nuestras tablas en la nube así como las variables de entorno generadas por la página para su conexión remota. Es aconsejable revisar las funciones sql que se empleen para tener un seguimiento de qué tablas son necesarias para la aplicación:



Y para *Render* podemos para mayor comodidad enlazar la cuenta del repositorio de *GitHub* con la de la página para acceder directamente a los archivos del proyecto, esta plataforma despliega de forma dinámica según los cambios introducidos en la rama principal del repositorio, actualizándose según vayamos desarrollando el código. Sin embargo es recomendable dejarlo para lo último debido a todas las alertas que da si el despliegue ha fallado o la aplicación ha dado un error. Este es el *Dashboard* del trabajo:



Capítulo 7

Pruebas

Tests creados específicamente para probar y comprobar de forma automatizada el correcto funcionamiento del proyecto y sus diferentes funciones.

7.1. Testing del backend

Parte del desarrollo software se basa en el ensayo y error de las funciones escritas, un método puede ser lo más sencillo y atómico posible haciendo que en teoría no presente ningún error, sin embargo en conjunto a otros en un programa con intercambio de variables asíncronas o en tiempo real o dependiendo de las salidas de otros métodos siempre puede producirse algún contratiempo o alguna instancia específica que pueda fallar el método.

Para ver la eficacia de los métodos, se han desarrollado una serie de test para comprobar el manejo correcto de las funciones según los distintos casos de uso o variables introducidas, permitiendo saber su eficacia sin tener que ejecutar manualmente las situaciones de error y automatizar el proceso a gran escala.

El testeo ha sido realizado con *JEST*, un framework de testing sencillo y flexible orientado a *JavaScript* compatible también con *NodeJS*. Ofrece múltiples cualidades a la hora de ejecutar pruebas del código implementado:

- Solución de testing de *JavaScript* completa y lista para instalación.
- Feedback instantáneo, los tests fallados son ejecutados primero. Modo interactivo rápido para cambiar entre ejecutar todos los tests o tests relacionados a solo los archivos cambiados.
- Testeo de snapshots: *Jest* puede capturar snapshots de árboles virtuales de React y otros valores serializables para simplificar el testeo de la UI.
- Capacidad de mockup para simular variables de sesión o los enlaces entre funciones del código, así como reproducir las variables de salida que pueden dar.

El proceso de testing se ha especializado según el tipo de dirección sin embargo se ha intentado seguir un criterio mínimo a la hora de probar las funciones de muestra de páginas. Vamos a clasificarlo en dos tipos:

- Devolución del *Status Code*: se comprueba el código html devuelto, ya sea correcto (**200**), encontrado (**302**), de error (**400**), no encontrado (**404**)...
- Devolución de la página correcta: las funciones de página pueden dar lugar a redirección o renderización de página comprobando que de lugar a la página específica que se da en esa situación.

```
describe('GET /test', () => {
  test('should respond with 200', async () => {
    const response = await request(app).get('/test').send();
    expect(response.statusCode).toBe(200);
  });
  test('should respond with json', async () => {
    const response = await request(app).get('/test').send();
    expect(response.headers['content-type']).toEqual(expect.stringContaining('json'));
  });
});
```

Figura 7.1: Testing de página de prueba

7.1.1. Testing de páginas

Vamos a probar la ejecución de cada página con tests individuales para cada una, recorriendo así los dominios creados para la herramienta web probando su correcto funcionamiento. Esto incluye las funciones de creación de usuario, login o incluso creación de video, se tuvo en cuenta que si el resultado de salida es de redirección los códigos de estatus no pueden establecer las cabeceras con los códigos normales dado que se envían antes de poder ser cambiadas, solo permite los códigos 302, 303 y 401.

7.1.1.1. Testing de página de inicio

Nos permite también si el server se inicia de forma correcta dado que es la página que se crea por defecto, no tiene parámetros de entrada así que comprobamos código (**200**) y que renderiza el archivo html (en este caso del fichero *portada.ejs*) de salida.

```
describe('GET /', () => {
  test('debería responder con 200', async () => {
    const response = await request(app).get("/").send();
    expect(response.statusCode).toBe(200);
  });
  test("debería responde con la página de la portada", async () => {
    const response = await request(app).get("/").send();
    expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
  });
});
```

Figura 7.2: Testing de página de portada

7.1.1.2. Testing de página de seguridad

Testing para la página de inicio de sesión.

```
describe('GET /seguridad', () => {
  describe('Yendo a la página de inicio de usuario', ()=>{
    test('debería responder con status code 200', async () =>{
      const response = await request(app).get('/seguridad').send();
      expect(response.statusCode).toBe(200);
    })
  })
  describe('Nos renderiza la página de inicio de usuario para identificarnos o ir al perfil', ()=>{
    test('debería responder con render', async () =>{
      const response = await request(app).get('/seguridad').send();
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
  describe('Nos da la página de seguridad con el inicio de sesión', ()=>{
    test('debería responder con render', async () =>{
      const response = await request(app).get('/seguridad').send();
      expect(response.text).toEqual(expect.stringContaining('Inicia sesión en HopBlog'));
    })
  })
});
```

Figura 7.3: Testing de página de inicio de sesión

7.1.1.3. Testing del lienzo

El lienzo nos tiene que devolver la página que contiene el elemento *Canvas* que se encuentra en el archivo *index.ejs*.

```
describe('GET /canvas', () => {
  test('debería responder con 200', async() => {
    const response = await request(app).get("/canvas").send();
    expect(response.statusCode).toBe(200);
  })
  test('debería responder con la página del lienzo', async () =>{
    const response = await request(app).get("/canvas").send();
    expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
  })
});
```

Figura 7.4: Testing de página del lienzo

7.1.1.4. Testing de página de usuario

Testing de página de usuario registrado, si no existe se redirige a la página de inicio de sesión.

```
describe('GET /usuario/nombre', () => {
  describe('Dado usuario existente: ', ()=>{
    const usuario = "Antonio";
    test('debería responder con status code 200', async () =>{
      const response = await request(app).get(`/usuario/${usuario}`).send();
      expect(response.statusCode).toBe(200);
    })
    test('debería responder con redirección a seguridad', async () =>{
      const response = await request(app).get(`/usuario/${usuario}`).send();
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
  describe('Dado usuario no existente en la base de datos: ', ()=>{
    const usuario = "Romualdo";
    test('debería responder con status code 302', async () =>{
      const response = await request(app).get(`/usuario/${usuario}`).send();
      expect(response.statusCode).toBe(302);
    })
    test('debería responder con redirección a seguridad', async () =>{
      const response = await request(app).get(`/usuario/${usuario}`).send();
      expect(response.text).toBe('Found. Redirecting to /seguridad');
    })
  })
});
```

Figura 7.5: Testing de usuario

7.1.1.5. Testing de previsualización de video

Testing para la página que muestra la previsualización antes de subir un video.

```
describe('GET /preview/:id/:nombre', () => {
  describe("Dado una id y un nombre: ", ()=>{
    const id = 333;
    const nombre = "video.mp4";
    test("debería responde con status code 200", async () =>{
      const response = await request(app).get("/preview/"+id+"/"+nombre).send();
      expect(response.statusCode).toBe(200);
    })
    test("debería responde con render a página de preview", async () =>{
      const response = await request(app).get("/preview/"+id+"/"+nombre).send();
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
});
```

Figura 7.6: Testing de preview

7.1.2. Testing de funciones

7.1.2.1. Testing de función de subida

Testing para la función de subida de videos a galería.

Emulamos los distintos estados de subir un vídeo a la plataforma, estaría localizado en la función de vista previa a subida, dependiendo de los parámetros que añadimos podemos probar la función de subida al servidor o cancelación y borrado de este, eliminando el ejemplo de las base de datos.

```
describe('POST /upload', () => {
  describe('Con id, nombre, etiquetas, no privado', ()=>{
    test("Añade video con status code 302", async () =>{
      const response = await request(app).post("/upload").send({
        botonform : 'Subir!',
        usuario : 'Antonio',
        vidid : 333,
        vidname : 'video.pm4',
        etiquetas : 'cara,ojo',
        privadocheck : 0
      })
      expect(response.statusCode).toBe(302);
    })
    test("Borra video con status code 302", async () =>{
      const response = await request(app).post("/upload").send({
        botonform : 'Borrar',
        usuario : 'Antonio',
        vidid : 333,
        vidname : 'video.pm4',
        etiquetas : 'cara,ojo',
        privadocheck : 0
      })
      expect(response.statusCode).toBe(302);
    })
  })
})
```

Figura 7.7: Testing de subida de videos

7.1.2.2. Testing de creación de usuario

Testing para la función de creación de un usuario en la base de datos.

```
describe('POST /creausr', () => {
  describe('Con nombre y contraseña nuevo', ()=>{
    test('Encuentra la página de inicio de usuario', async () =>{
      const response = await request(app).post("/creausr").send({
        user_email : 'ejemplo',
        user_password : 'contra',
      })
      expect(response.statusCode).toBe(200);
    })
    test('Redirige a la página de usuario', async () =>{
      const response = await request(app).post("/creausr").send({
        user_email : 'ejemplo',
        user_password : 'contra',
      })
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
  describe('Con nombre repetido', ()=>{
    test('Recibe mensaje de error', async () =>{
      const response = await request(app).post("/creausr").send({
        user_email : 'Antonio',
        user_password : 'aaa',
      })
      expect(response.statusCode).toBe(200);
    })
    test('Muestra mensaje de error', async () =>{
      const response = await request(app).post("/creausr").send({
        user_email : 'Antonio',
        user_password : 'aaa',
      })
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
})
```

```

describe("Sin nombre ni contraseña", ()=>{
  test("Recibe mensaje de error", async () =>{
    const response = await request(app).post("/creausr").send({
      user_email: undefined,
      user_password: undefined,
    })
    expect(response.statusCode).toBe(200);
  })
  test("Muestra mensaje de error", async () =>{
    const response = await request(app).post("/creausr").send({
      user_email: undefined,
      user_password: undefined,
    })
    expect(response.text).toBe('introduce mail y contraseña');
  })
});

```

Figura 7.8: Testing de creación de usuario

7.1.2.3. Testing de login

El sistema de login tiene varias posibles salidas, dependiendo de si tenemos usuario correcto o no damos un parámetros a la hora de introducir usuario. Aquí tenemos los distintos casos siendo el código 401 de no autorizado y 302 de encontrado:

En caso de parámetro incorrecto redirección con mensaje de 'usuario incorrecto'.

```

describe("POST /login", () => {
  describe("Dado nombre y contraseña", () => {
    test("debería responder con status code 302 found", async () => {
      const response = await request(app).post("/login").send({
        user_email: "Antonio",
        user_password: "contrasenia"
      })
      expect(response.statusCode).toBe(302);
    })
    test("debería responder con redirección", async () => {
      const response = await request(app).post("/login").send({
        user_email: "Antonio",
        user_password: "contrasenia"
      })
      expect(response.text).toBe("Found. Redirecting to /seguridad");
    })
  })
})

describe("Dado nombre incorrecto y contraseña", () => {
  test("debería responder con status code 401 no autorizado", async () => {
    const response = await request(app).post("/login").send({
      user_email: "noexiste",
      user_password: "contrasenia"
    })
    expect(response.statusCode).toBe(401);
  })
  test("debería responder con redirección", async () => {
    const response = await request(app).post("/login").send({
      user_email: "noexiste",
      user_password: "contrasenia"
    })
    expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
  })
})

describe("Dado nombre y contraseña incorrecto", () => {
  test("debería responder con status code 401 no autorizado", async () => {
    const response = await request(app).post("/login").send({
      user_email: "Antonio",
      user_password: "contraseniafalsa"
    })
    expect(response.statusCode).toBe(401);
  })
  test("debería responder con redirección", async () => {
    const response = await request(app).post("/login").send({
      user_email: "Antonio",
      user_password: "contraseniafalsa"
    })
    expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
  })
})

describe("Dado contraseña y no nombre", () => {
  test("debería responder con status code 401", async () => {
    const response = await request(app).post("/login").send({
      user_email: "Antonio",
      user_password: undefined
    })
    expect(response.statusCode).toBe(401);
  })
  test("debería responder con redirección", async () => {
    const response = await request(app).post("/login").send({
      user_email: "Antonio",
      user_password: undefined
    })
    expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
  })
})

describe("Dado contraseña y no nombre", () => {
  test("debería responder con status code 401", async () => {
    const response = await request(app).post("/login").send({
      user_email: undefined,
      user_password: "contrasenia"
    })
    expect(response.statusCode).toBe(401);
  })
  test("debería responder con redirección", async () => {
    const response = await request(app).post("/login").send({
      user_email: undefined,
      user_password: "contrasenia"
    })
    expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
  })
})

describe("Si no se da nombre y contraseña", () => {
  test("debería responder con status 402", async () => {
    const response = await request(app).post("/login").send({
      user_mail_address: undefined,
      user_password: undefined
    })
    expect(response.statusCode).toBe(401);
  })
  test("debería responder con texto", async () => {
    const response = await request(app).post("/login").send({
      user_mail_address: undefined,
      user_password: undefined
    })
    expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
  })
})

```

7.1.2.4. Testing de logout

Logout es una función de eliminación de almacenamiento de sesión, redirigiendo a la página de login original.

```
describe('GET /logout', () => {
  describe("Yendo a la página de inicio de usuario", ()=>{
    test("debería responde con status code 302", async () =>{
      const response = await request(app).get("/logout").send();
      expect(response.statusCode).toBe(302);
    })
  })
  describe("Nos da la página de seguridad con el inicio de sesión", ()=>{
    test("debería responder con render", async () =>{
      const response = await request(app).get("/logout").send();
      expect(response.text).toBe("Found. Redirecting to /seguridad");
    })
  })
});
```

Figura 7.9: Testing de página de prueba

7.1.2.5. Testing de galería y búsqueda en la galería

Necesitamos comprobar si la galería de videos muestra por pantalla el archivo .ejs de galería de videos. También comprobamos si la petición es a la función de búsqueda por etiquetas si nos da un resultado de la página:

```
describe('GET /muestra3', () => {
  describe("Accediendo a la galería", ()=>{
    test("debería responde con status code 200", async () =>{
      const response = await request(app).get("/muestra3").send();
      expect(response.statusCode).toBe(200);
    })
    test("debería responde con la página html con los resultados", async () =>{
      const response = await request(app).get("/muestra3").send();
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
});
```

Figura 7.10: Testing de página de galería

```
describe('POST /muestrabusqueda', () => {
  describe("Dada etiqueta", ()=>{
    test("debería responde con status code 200", async () =>{
      const response = await request(app).post("/muestrabusqueda").send({
        etiquetasbuscar : 'cara,ojo'
      });
      expect(response.statusCode).toBe(200);
    })
    test("debería responde con la página html con los resultados", async () =>{
      const response = await request(app).post("/muestrabusqueda").send({
        etiquetasbuscar : 'cara,ojo'
      });
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
});
```

Figura 7.11: Testing de función de búsqueda en la galería

7.1.2.6. Testing de visión de administrador y borrado de videos

El administrador tiene acceso a una galería total del servicio, donde puede ver todos los subidos que hayan sido subidos a la página sean privados o no, este además puede borrar los vídeos si así lo considera con al función de borrado de video.

```
describe('GET /totalAdmin', () => {
  describe("Accediendo a la galería", ()=>{
    test("debería responde con status code 200", async () =>{
      const response = await request(app).get("/totalAdmin").send();
      expect(response.statusCode).toBe(200);
    })
    test("debería responde con la página html con los resultados", async () =>{
      const response = await request(app).get("/totalAdmin").send();
      expect(response.headers['content-type']).toEqual(expect.stringContaining("html"));
    })
  })
});
```

Figura 7.12: Testing de galería total

```
describe('POST /borraavid', () => {
  describe("Dado id de video", ()=>{
    test("debería responde con status code 302", async () =>{
      const response = await request(app).post("/borraavid").send({
        idvideoborrar : '333.mp4'
      });
      expect(response.statusCode).toBe(302);
    })
    test("debería responder con redirección a /totalAdmin", async () =>{
      const response = await request(app).post("/borraavid").send({
        idvideoborrar : '333.mp4'
      });
      expect(response.text).toBe("Found. Redirecting to /totalAdmin");
    })
  })
});
```

Figura 7.13: Testing de borrado de galería

Normalmente nos dará una salida como error al testear este módulo debido a que estamos borrando un video que no existe se podría evitar este error cambiando el orden poniendo este test después del test de simulación de subida de video sin embargo hay más funciones de borrado que de subida implementadas.

Capítulo 8

Conclusiones y trabajos futuros

Esta sección está dedicada a la discusión y valoración personal de las tareas y objetivos autoimpuestas en el trabajo, así como posibles mejoras para el desarrollo de este en otras condiciones o con la información aprendida a medida que se ha ido desarrollando el software a lo largo de los meses. También se comparará con otros software del mercado con objetivo similares ya habiendo desarrollado este, viendo en retrospectiva qué se podría haber tomado o cambiado ahora con más conocimiento del mecanismo interno de funcionamiento de una aplicación del estilo.

8.1. Conclusiones

El objetivo principal de este proyecto era recrear en un navegador web el programa *Flipnote Studio*, es decir una herramienta sencilla de animación con las herramientas justas para usar de forma casual. Desde el principio no se intentó llevar a cabo esta idea de forma exhaustiva, el objetivo no era hacer un port o reproducción exacta del software sino una alternativa para ordenadores que permita en cierta medida replicar la experiencia de uso del juego; que permita tanto a los nostálgicos que busquen esa misma sensación volver a poder tenerla y a los usuarios nuevos que prueben algo que ya no se puede probar a día de hoy; una forma de preservar el pasado dado que no se puede conseguir a día de hoy de forma oficial por la empresa.

Hablando como desarrollador y principal público de este proyecto, siento que he conseguido realizar una herramienta que pueda sentirse intuitiva y no requiera de instalación para su uso, un software de animación que pueda ejecutarse en cualquier máquina sin apenas preparación para sesiones cortas, una actividad lúdica para intervalos de tiempo corto. Sin embargo sí es cierto que el proyecto puede alejarse de la idea preconcebida originalmente al empezarlo.

8.2. Tareas cumplidas

Disponíamos en el capítulo 2 (sección 2.3) de una lista de objetivos a cumplir para el proyecto antes de haber empezado tan siquiera la planificación de los sprints, estos objetivos se dividían entre los dos requerimientos del proyecto: La capacidad de dibujar y crear animaciones de forma contenida en un navegador web y el soporte del software de forma remota gracias a un servidor.

8.2.0.1. Objetivos de aplicación

- **OA1:** El producto es accesible desde cualquier navegador, dado que está alojado en una plataforma web de despliegue en la red. Se ha empleado los servicios de *Render* para el despliegue de la aplicación en la nube, enlazado directamente con el repositorio en *Github* así como la base de datos está alojada en *Clever Cloud*.
- **OA2:** Se puede dibujar con trazo libre en un elemento *Canvas* de html y mediante las funciones de rasterización de píxeles marcados dentro del recinto y registro de la posición del ratón al pulsar el lienzo. Gracias a las funciones de *JavaScript* nativas con respecto al elemento *Canvas* podemos simular trazo libre dentro de un lienzo uniendo pixel a pixel del contenido 2d del lienzo con las funciones de `context.lineTo()`, `context.lineCap = 'round'`, `context.lineJoin = 'round'` y `context.stroke()`.
- **OA3:** Se pueden crear varios lienzos consecutivos con un número de fotograma asignado para su posterior guardado y procesado como video, creando así varios fotogramas. Dado que el contenido de un lienzo html se puede guardar como variable, se crea un array de estos los cuales vamos vaciando y cargando en el lienzo visibles según nos movemos hacia delante y atrás en los fotogramas dibujados de la página.
- **OA4:** Se ha creado una galería con todos los videos subidos a la plataforma web, es decir un acceso al contenido en video de la base de datos mostrando las animaciones del usuario y las de otros de la comunidad. Se ha creado una página del servidor que accede a la base de datos y filtra aquellas animaciones privadas o descartadas por el criterio de búsqueda introducido, esto es trabajo del servidor para filtrar aquellas entradas de la base de datos de video que no encajen con las peticiones, variando con cada usuario.
- **OD1:** Desde el servidor lanzado por *NodeJS* se puede conectar a la base de datos de *MySQL* y realizar peticiones de inserción y filtrado de los datos que contiene. Gracias al módulo de *Express* podemos realizar peticiones sql a nuestra base de datos para manejar la información con un almacenamiento estático y seguro para guardarlo, además gracias a la plataforma

de *Render* se ha podido desplegar la aplicación web para su uso remoto desde cualquier ordenador.

- **OD2:** Se han creado los roles de usuario normal y administrador para la creación de animaciones así como ejercer de moderación de contenido por parte de los administradores con privilegios de visibilidad del contenido de otros usuarios y capacidad de borrado de la base de datos. Añadiendo un parámetro más a la tabla de usuarios de la base de datos correspondiente se puede trabajar en base a ese valor añadiendo y dando funcionalidades nuevas a los distintos tipos de rol dentro de la página.
- **OD3:** Se ha creado un sistema de envío de frontend a backend de fotogramas dibujados para su guardado y renderizado en forma de video ordenado para su posterior consulta en la galería. Mediante la automatización del proceso de envío a servidor y a su vez en la base de datos, podemos gracias al framework multimedia de *FFmpeg* crear un video de las imágenes generadas de dibujar en el lienzo que ve el cliente.
- **OD4:** Se ha creado una interfaz de usuario para indicar las herramientas disponibles e información necesaria en las instancias de caso de uso descritas en el proyecto. Mediante css y páginas de diseño html, como *Colorpicker* y similares se ha intentado hacer un formato de diseño uniforme e intuitivo con botones con iconografía para describir sus funciones, sin embargo el acabado es un poco amateur y podría haber necesitado o una capa de abstracción en la parte del cliente para facilitar una página interactiva mínima o mayor idea de diseño gráfico a la hora de crear el display en pantalla.
- **OD5:** Se ha creado un ambiente estable de trabajo y sin demasiados problemas técnicos o de lógica de funcionamiento. Los frameworks de trabajo y módulos de testeo implementados como *Express*, *Nodemon* o *Jest* permiten comprobar el correcto funcionamiento del software así como el envío de información y respuesta correcta de los parámetros; aunque algún error presenta por la parte del código del frontend debido a la inestabilidad de las funciones *fetch* para envío de información recurrente o *EventListener()* que han podido no reaccionar del modo que uno esperaba según una secuencia de modo de uso específica; pudiendo dar la sensación de inestable o de búsqueda de un método más resiliente.

En general este proyecto tenía ideas factibles a desarrollar sin embargo no muchas páginas del mercado actual ofrecen la posibilidad de un servicio similar de forma gratuita o al menos de forma sencilla para llegar a más público; muchos que he ido encontrando han sido proyectos similares pero abandonados o simplemente ignorando el material original del que se parte. Respecto al desarrollo el trabajo ha sido una actividad autodidacta de la tecnología detrás

del procesado de imágenes en la nube así como el funcionamiento del paso de información y codificación de valores entre cliente y servidor, planteando la duda de si en caso de haber empleado otro lenguaje se hubiera implementado de forma distinta con respecto al trato de envío, pudiendo dar lugar a otras características de la aplicación ¿Quizás implementación en móviles? o una interfaz más modular barajando la posibilidad de un desarrollo de una herramienta de *Single-page application*. Este proyecto cuenta con licencia general pública para siempre poder volver a este proyecto por el autor o cualquiera interesado.

8.3. Conclusiones del estado del arte

Habiendo dedicado un extenso periodo de tiempo al desarrollo de una aplicación de dibujo se puede entender mejor la tecnología detrás de los servicios actuales de dibujo o animación actuales. Este es un proyecto modesto pero no tiene por qué distanciarse en metodología de software de sobremesa como *ClipStudio Paint* en cuanto a cómo funcionan ciertas herramientas, haciendo que un desarrollador se plantee cuál es la solución más fácil para hacer una funcionalidad específica o hacer que parezca que tiene el mismo resultado sin hacer la misma operación; poniendo de ejemplo el sistema de borrado puede simplemente ser cambiar el color de trazo al mismo del fondo dando la ilusión de borrado y a la vez dando la capacidad de retroceso de la acción de forma sencilla sin tener que sobrepensar la lógica detrás de cómo borrar trazos de un dibujo. El formato *PNG* ha sido el más conveniente para el uso de imágenes renderizadas en el proyecto, su fácil renderización con la información en base 64 de los fotogramas guardados ha permitido la creación de estas de forma automática y en cadena en el servidor. Este formato además es el nativo que emplea *FFmpeg* para la creación de videos, este framework es sumamente útil para el desarrollo multimedia en *JavaScript* y podría haberse aprovechado más sus funciones asignadas al codificador *h.264* como la de velocidad de reproducción o control del bitrate.

Si hay alguna propuesta actual que cumpla las mismas características de mejor forma es sin lugar a dudas *Gartic Phone*; como comentamos en la sección 3.2 es una página orientada al entretenimiento que tiene múltiples modos de dibujo así como uno de animar en grupo. No soy consciente de si tienen algún tipo de galería pública debido a que no guarda registro de quienes son usuarios con sesión iniciada o no sin embargo su trabajo a la hora de generar animaciones es remarcable y podría haberse aprendido más de su método de trabajo así como de su presentación e interfaz gráfica.

8.4. Trabajos futuros

Respecto a los trabajos futuros este proyecto surge de un deseo de querer preservar la herramienta de *Flipnote Studio* así que siempre se querría más

funciones similares a las del programa original. Múltiples herramientas fueron descartadas por varios motivos, algunas como la implementación del micrófono o efecto 3D no son más que funcionalidades implementadas por las capacidades de las consolas portátiles originarias que no cambian mucho la experiencia de uso; sin embargo cosas como un sistema de capas semitransparentes o papel de cebolla para ver el frame anterior siempre es una ventaja para este tipo de software.

Más herramientas para mejorar la calidad de experiencia como pinceles con patrones, cubos de pintura para rellenar figuras, estampados o selección de regiones del lienzo son funcionalidades del programa original que con tiempo extra se querrían haber implementado.

Dentro de la lógica de base de datos se podría mejorar el sistema de identificación de fotogramas, dando lugar a reconocimiento unívoco de imágenes sin tener que emplear tantos dígitos o caracteres.

Con mejor entendimiento sobre el desarrollo web y desarrollo frontend, se querría revisar alternativas para el paso de información entre servidor y cliente y viceversa para mejorar el envío de fotogramas y procesado en el envío al servidor.

Una nueva interfaz o diseño gráfico vendría bien para atraer a más público o hacer un portal web más accesible para los usuarios, encontrar un diseño definitorio para la herramienta que pueda hacer que destaque.

Capítulo 9

Apéndice

Capítulo dedicado al método de empleo de la aplicación. Veremos cómo acceder a ella y su método de uso.

9.1. Cómo acceder a *Proyecto Hopblog*

Esta memoria está localizada junto al código de ejecución de *Proyecto Hopblog*, a continuación se detalla dos formas de ejecutar el código, una a través del navegador como debería emplearse como usuario de la página y otra descargando el código de ejecución para su despliegue en red local.

9.1.1. Acceso por navegador web

Gracias a la plataforma de *Render* se ha podido desplegar el proyecto en la nube, con la facilidad añadida de cualquier actualización realizada a la rama principal de su repositorio correspondiente se verá reflejado en la página por su despliegue automático enlazado a *Github*.

Se encuentra alojada en la dirección url de <https://proyecto-hopblog.onrender.com/>, donde veremos la página principal de portada con una breve explicación del proyecto, listo para su uso. Hay que tener en cuenta que a cada actualización sobre el despliegue del proyecto borrará los vídeos almacenamos debido a que no forman parte de la rama principal del código original.

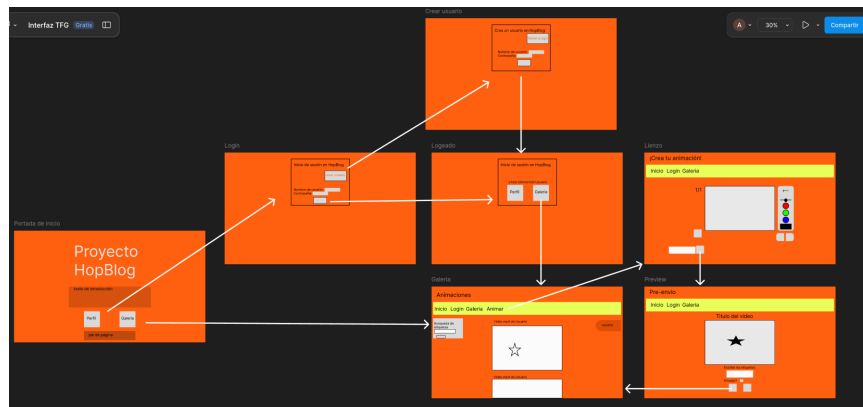
9.1.2. Descarga de repositorio

Ahora si accedemos al repositorio del proyecto [2] podemos descargar el código como archivo comprimido zip o clonar y crear la rama de *Git* dentro de nuestra máquina con `git clone https://github.com/ChinChainis/Proyecto-Hopblog.git`. Una vez instalado el código, con la ejecución de `npm start` se bajarán todas las dependencias necesarias para ejecutar y se desplegará en una

dirección local de nuestra red wifi el código. Revisar las variables de entorno en caso de fallo.

9.2. Método de uso

El diseño de las páginas tiene la intención de siempre poder saltar de unas a otras en caso de querer realizar una acción o volver a la anterior, para ello seguimos el siguiente esquema de cómo acceder a las distintas vistas:

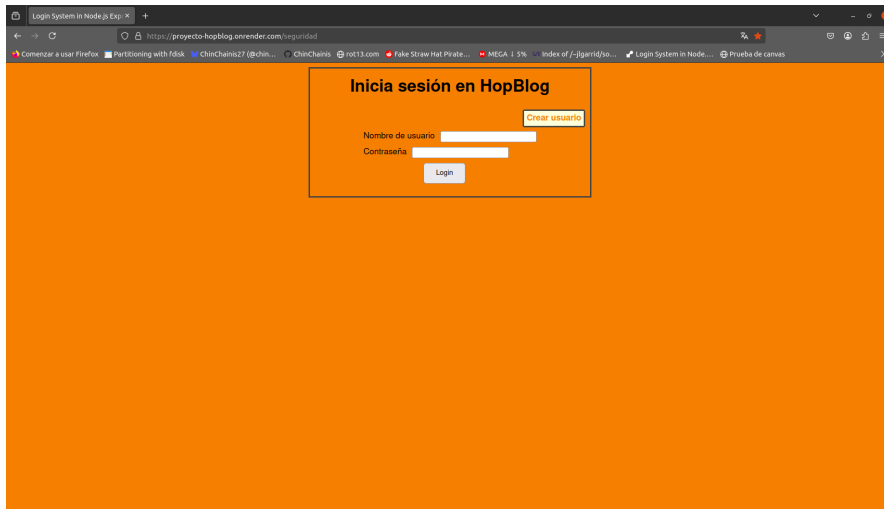


Empezando por la portada, tiene una breve descripción del proyecto y sus acreditaciones. De aquí se puede ir a iniciar sesión en el apartado de **Perfil** o a la galería de animaciones de **Galería**.

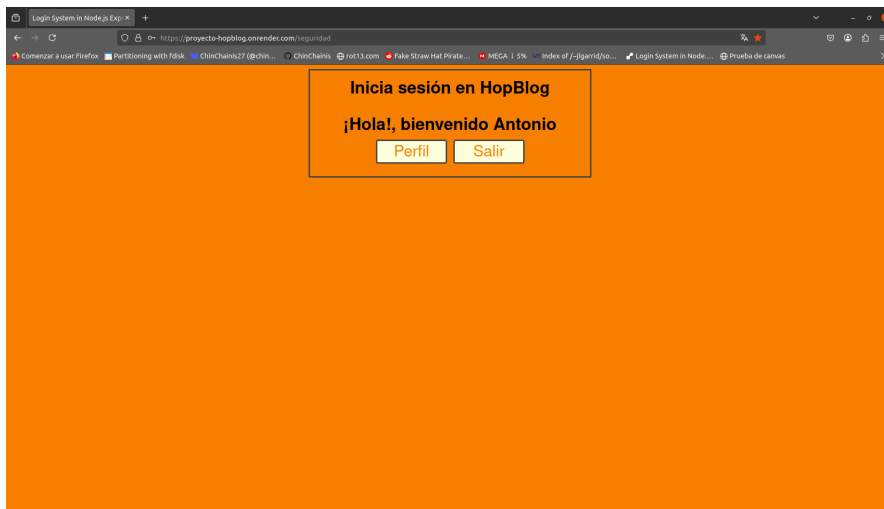


Se puede acceder a la galería sin haber iniciado sesión, sin embargo vamos a iniciar para ver todas las opciones que disponemos con esta. Podemos tanto iniciar con un usuario registrado como crear uno para tener las animaciones que

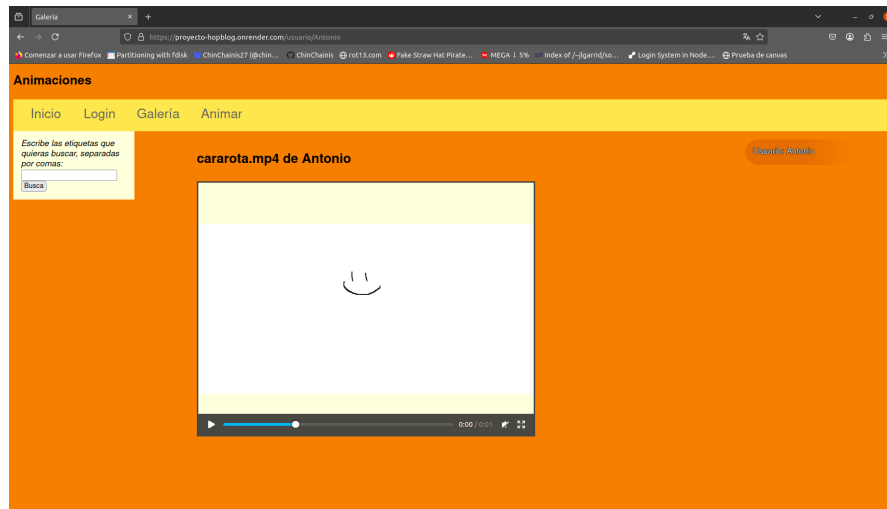
creamos a nuestro nombre. No se puede crear un usuario con nombre repetido y las credenciales deben estar bien escritas:



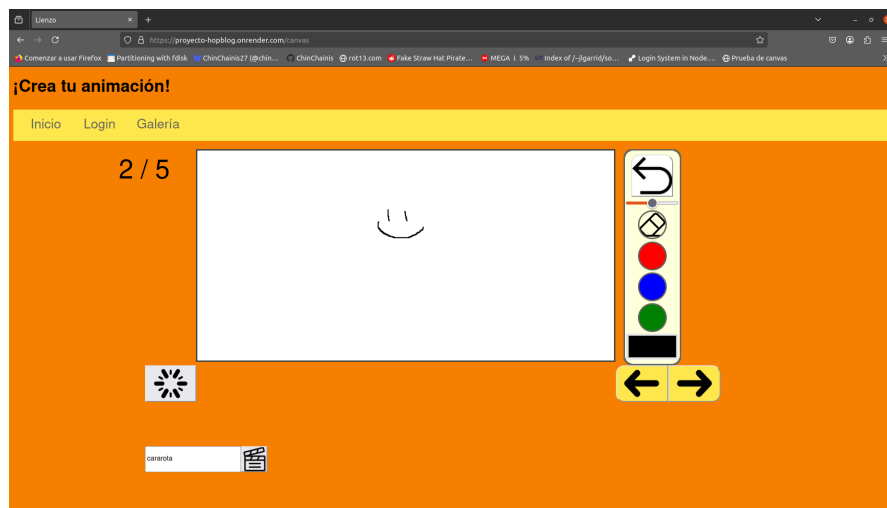
Una vez iniciado sesión, se mantendrá a menos que recarguemos las cookies del sitio, se identificará el usuario creado cuando veamos la galería u otras páginas de visionado de animaciones:



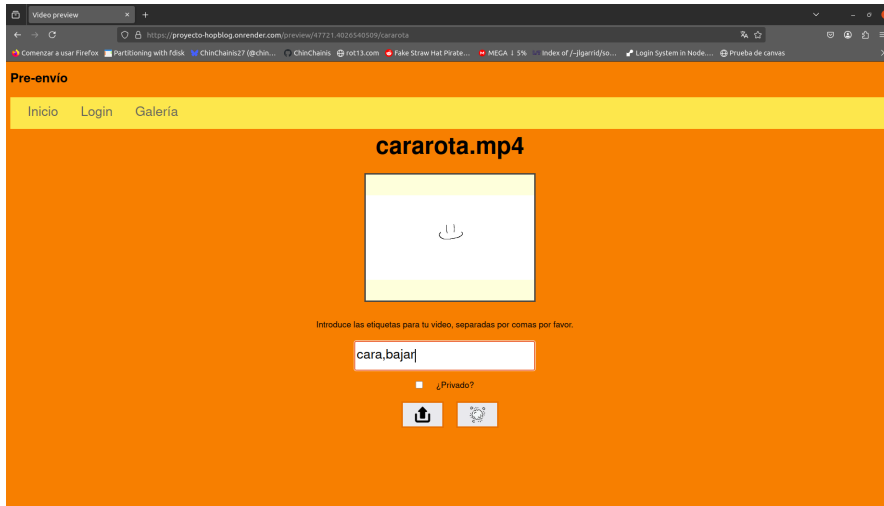
Nos redirige a nuestro usuario con las animaciones almacenadas en la página a nuestro nombre, habiendo iniciado sesión tenemos acceso a múltiples animaciones a nuestro nombre si hemos creado anteriormente:



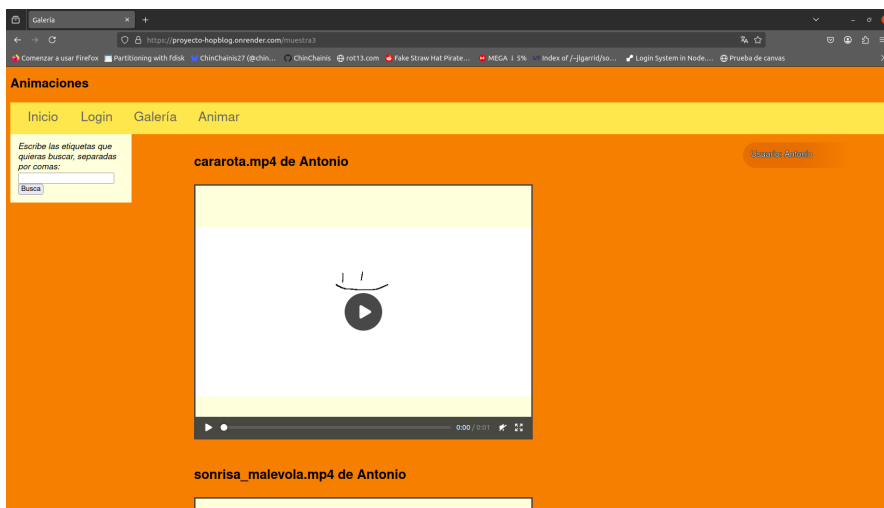
Vamos a crear una animación de ejemplo, vamos a la opción de animar y tenemos nuestro lienzo para dibujar, ponemos título a nuestra animación y damos tiempo para renderizar el video:



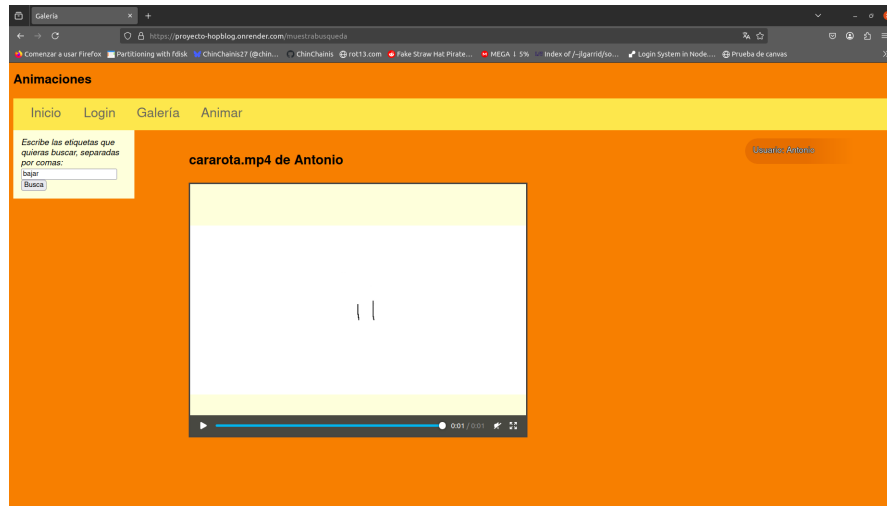
Habiendo cargado la página y el vídeo, tenemos una previsualización de la animación a subir. Aquí podemos darle etiquetas, ponerlo en estado de privado o no para que sea visible por otros usuarios o directamente descartar el video si no estamos satisfechos con él.



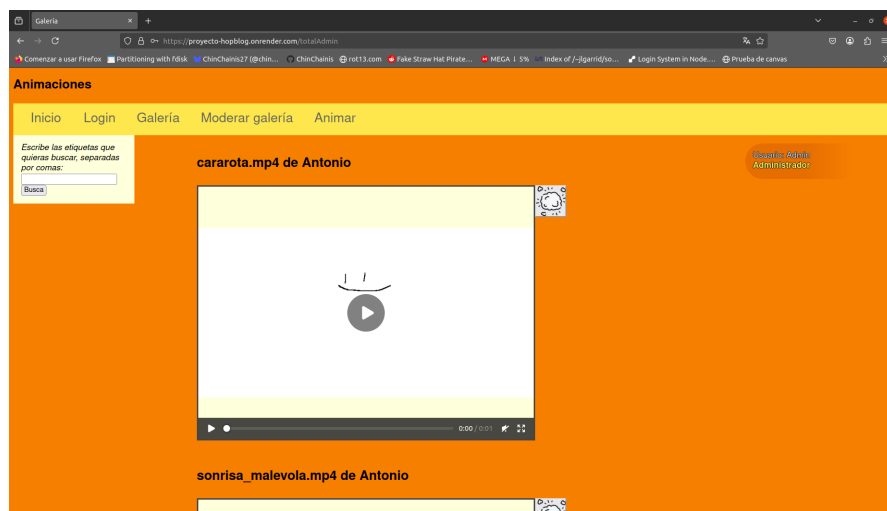
Aquí vemos la galería completa con todas las animaciones subidas, podemos ver la que acabamos de crear junto a otras:



Como hemos registrado la animación reciente con etiquetas, podemos buscarla en el buscado del margen izquierdo. Aquí podemos filtrar animaciones tanto por etiquetas como por autores:



Ahora, si un usuario con el rol de administrador decide que la animación no es adecuada para las normas de uso de la página, puede eliminarlo de la base de datos gracias al botón a la derecha de cada vídeo. Método solo accesible para administradores.



Bibliografía

- [1] Carrillo León Antonio Luis Cejudo López José Manuel Domínguez Muñoz Fernando Rodríguez García Eduardo A. Graphics tablet technology in second year thermal engineering teaching. <https://upcommons.upc.edu/handle/2099/15765>. 17/6/2025.
- [2] Antonio Sánchez Alcaraz. Repositorio de proyecto-hopblog. <https://github.com/ChinChainis/Proyecto-Hopblog>. 17/2/2025.
- [3] Kent Beck, Mike Beedle, Arie Van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, et al. Manifesto for agile software development. https://ai-learn.it/wp-content/uploads/2019/03/03_ManifestoofAgileSoftwareDevelopment-1.pdf, 2001. 20/8/2025.
- [4] Matic Broz. Image format comparison: Jpeg vs. png vs. webp vs. avif. <https://photutorial.com/image-format-comparison-statistics/>. 28/8/2025.
- [5] Calcium. About clipnote studio. <https://calcium-chan.itch.io/clipnote>. 18/7/2025.
- [6] Patrick Davison. Because of the pixels: On the history, form, and influence of ms paint. <https://journals.sagepub.com/doi/full/10.1177/1470412914544539>. 15/7/2025.
- [7] Clever Cloud dev team. Clever cloud | fair and simple pricing. <https://www.clever-cloud.com/pricing/>. 31/8/2025.
- [8] Eric Drowell. Kineticjs. <https://github.com/ericdrowell/KineticJS>. 28/8/2025.
- [9] Glassdoor.es. Sueldos para el puesto de programador web en granada. <https://www.glassdoor.es/Sueldos/>

- [granada-programador-web-sueldo-SRCH_IL.0,7_IC2614045_K08,23.htm](#). 31/8/2025.
- [10] Internet Janitor. Wigglypaint. <https://internet-janitor.itch.io/wigglypaint>. 18/7/2025.
- [11] Seo James Jung-Hoon. Interactive cinema : collaborative expression with digital video. <https://dspace.mit.edu/handle/1721.1/62358>. 31/5/2025.
- [12] lavtron. konvajs. <https://github.com/konvajs/konva>. 28/8/2025.
- [13] Antonio Horno López. El storyboard o guión gráfico. <https://www.ugr.es/~ahorno/STA.pdf>.
- [14] Mack. The history of digital drawing software. <https://www.ensemble.art/blog/the-history-of-digital-drawing-software>. 6/7/2025.
- [15] Rafael Molina. Tema 11: Estándares de compresión de imágenes. Apuntes de la asignatura de Compresión y Recuperación de Información Multimedia. 17/8/2025.
- [16] Rafael Molina. Tema 7: Conceptos para la compresión con pérdida. Apuntes de la asignatura de Compresión y Recuperación de Información Multimedia. 17/8/2025.
- [17] Múltiples. All about images. <https://guides.lib.umich.edu/c.php?g=282942&p=1885352>. 15/7/2025.
- [18] Múltiples. Artículo sobre el uso educativo de flipnote studio. https://conference.pixel-online.net/conferences/ICT4LL2011/common/download/Paper_pdf/IEL07-234-FP-Munro-ICT4LL2011.pdf. 8/4/2025.
- [19] Múltiples. Blender's history. <https://www.blender.org/about/history/>. 7/7/2025.
- [20] Múltiples. Clipstudio main page. <https://www.clipstudio.net/en/>. 15/7/2025.
- [21] Múltiples. Créditos sudomemo. <https://support.sudomemo.net/credits/>. 18/7/2025.
- [22] Múltiples. Definición de software libre - wikipedia, la enciclopedia libre. https://es.wikipedia.org/wiki/Definici%C3%B3n_de_software_libre. 17/6/2025.
- [23] Múltiples. Gartie telephone guide. <https://www.liverpool.ac.uk/researcher/postdoc-appreciation-week/npdc/engagement/pre-conference/gartie-telephone/>. 18/7/2025.

- [24] Múltiples. H.264 : Codificación de vídeo avanzada para los servicios audio-visuales genéricos. <https://web.archive.org/web/20210727123623/https://www.itu.int/rec/T-REC-H.264/es>. 28/8/2025.
- [25] Múltiples. Renta web open simulador. <https://www2.agenciatributaria.gob.es/wlpl/PARE-RW24/OPEN/index.zul?TACCESO=COLAB&EJER=2024>. 31/8/2025.
- [26] phuc.es. Jornada laboral en españa 2025: Cálculo y distribución de las horas laborales 2025. <https://phuc.es/horas-laborales-2025/>. 31/8/2025.
- [27] render.com. Predictable pricing that scales with you. <https://render.com/pricing>. 1/9/2025.
- [28] Iain E. Richardson. The h.264 advanced video compression standard. https://books.google.es/books?hl=es&lr=&id=k7n0AiIUo9IC&oi=fnd&pg=PR13&dq=h.264&ots=rGWkltOGNK&sig=4Gfoc0_XX1VUtMB8noLn27rVJOM#v=onepage&q=h.264&f=false. 28/8/2025.
- [29] Jan Wedekind. 12: Encoding videos using ffmpeg | download scientific diagram. https://www.researchgate.net/figure/Encoding-videos-using-FFmpeg_fig10_308025741. 28/8/2025.
- [30] Hao-Ping (Hank) Lee Advait Sarkar Lev Tankelevitch Ian Drosos Sean Rintel Richard Banks Nicholas Wilson. The impact of generative ai on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. <https://dl.acm.org/doi/full/10.1145/3706598.3713778>. 18/6/2025.